Lecture 1
Introduction to Operating Systems
19CSE214 : OPERATING SYSTEMS

Contents

- Course Overview
- Syllabus
- Text books and References
- Course outcomes and CO-PO Mapping
- Materials
- Lecture-1

Course Overview

- This course aims at introducing the structure and implementation of modern operating systems, virtual machines and their applications.
- It summarizes techniques for achieving process synchronization and managing resources like memory, CPU, and files and directories in an operation system.
- A study of common algorithms used for both pre-emptive and non-pre-emptive scheduling of tasks in operating systems (such a priority, performance comparison, and fair-share schemes) will be done.
- It gives a broad overview of memory hierarchy and the schemes used by the operating systems to manage storage requirements efficiently.

Course Outcomes

- **CO1**: Understand the architecture and system calls of OS
- **CO2**: Understand and apply the algorithms for resource management and scheduling.
- **CO3**: Analyze and Apply semaphores and monitors for classical and real-world synchronization scenarios.
- **CO4**: Engage in independent learning as a team and implement OS functionalities on a simulated machine.

CO-PO Mapping

PO/PSO / CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	2	1	1		2								3	2
CO2	2	2	3	1	3								3	2
CO3	2	3	3	2									3	2
CO4	2	2	1	2	3				2				3	2

Syllabus 3-0-2-4

Unit 1

- **Operating systems Services**
- Overview – hardware protection, services, system calls, system structure, virtual machines. Process and Processor management. Process concepts, process creation, process scheduling, operations on process, cooperating process, inter-process communication. Multi-threading models – threading issues, thread types. CPU scheduling – scheduling algorithms.

Unit 2

- **Process and Memory Management**
- Process synchronization- critical section problem, synchronization hardware, semaphores, classical problems of synchronization, critical regions, monitors. Deadlocks – deadlock characterization, methods of handling deadlocks, deadlock prevention, avoidance, detection and recovery. Memory management - swapping, contiguous memory allocation, paging and segmentation, segmentation with paging, virtual memory, demand paging, page replacement, thrashing.

Unit 3

- **File and Disk Management**
- File management - File systems, directory structure, directory implementation. Disk Management – storage structure, disk scheduling.
- Case study: Implement OS functionalities on a simulated machine (For example: XOS or eXperimental Operating System)

- **Textbook(s)**
- *Silberschatz A, Gagne G, Galvin PB. "Operating system concepts". Tenth Edition, John Wiley and Sons; 2018.*
- **Reference(s)**
- *Andrew S. Tanenbaum, Herbert Bos, "Modern Operating Systems", Pearson Education India; Fourth edition 2016.*
- *Stevens WR, Rago SA. "Advanced programming in the UNIX environment". Second Edition, Addison-Wesley; 2013.*

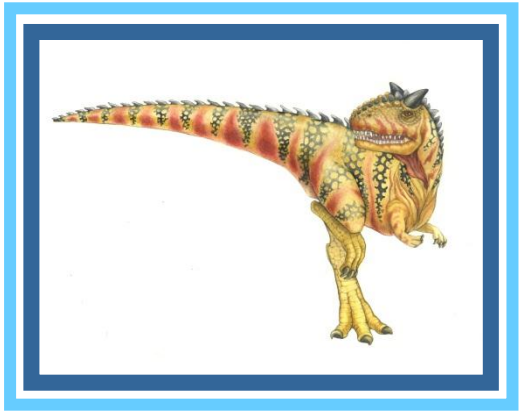
Evaluation Pattern: 70:30

Assessment	Internal	End Semester
Midterm	20	
*Continuous Assessment (Theory) (CAT)	10	
*Continuous Assessment (Lab) (CAL)	40	

**End Semester		30 (50 Marks; 2 hours exam)

*CA – Can be Quizzes, Assignment, Case-study Reports.
**End Semester can be theory examination/ lab-based examination

Chapter 1: Introduction



Unit 1

Operating systems Services

Overview – hardware protection, services, system calls, system structure, virtual machines. Process and Processor management. Process concepts, process creation, process scheduling, operations on process, cooperating process, inter-process communication. Multi-threading models – threading issues, thread types. CPU scheduling – scheduling algorithms.

Objectives

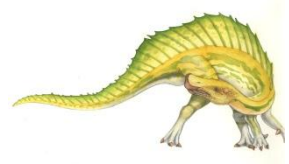
- To describe the basic organization of computer systems
- To provide a grand tour of the major components of operating systems
- To give an overview of the many types of computing environments
- To explore several open-source operating systems

What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

The operating system controls the hardware and coordinates its use among the various application programs for the various users.

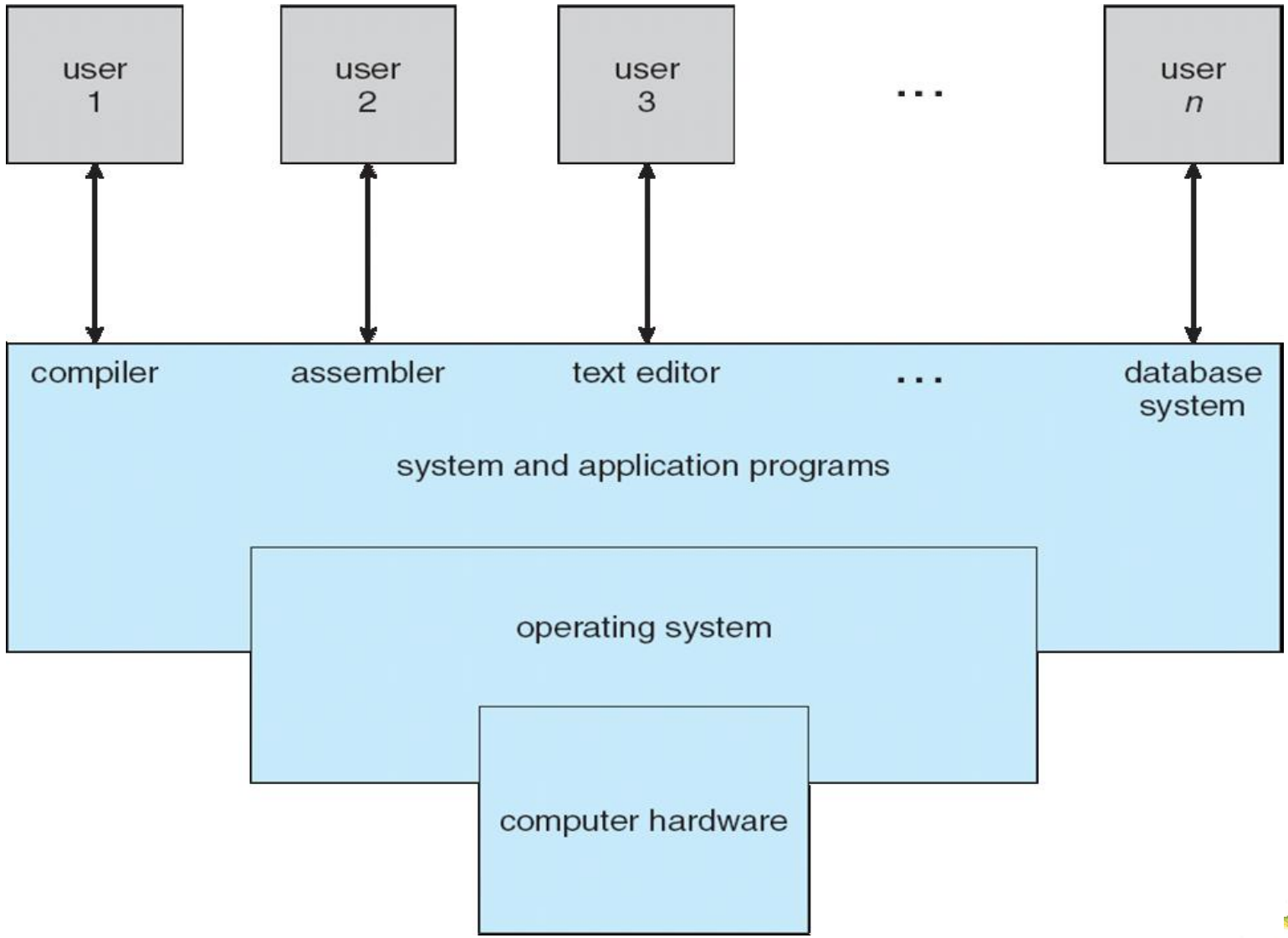
It simply provides an environment within which other programs can do useful work.



Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, web browsers, database systems, video games
  - Users
    - People

Four Components of a Computer System



- Exploring operating system from two view points

1) User View

2) System View

What Operating Systems Do

- **User View**
- Users(Single User) want convenience, **ease of use** and **good performance**

Don't care about **resource utilization(how various hw and sw resources are shared). This type of system is optimized for single user experience rather than the requirements of multiple users**

From the user's perspective, ***the OS is simply a tool that makes the system easy to use.***

- But shared computer such as **mainframe** or **minicomputer** must keep all users happy. Resource utilization is important. All resources are shared fairly among the users. **Example: z/OS**

OS focuses more on fairness and efficiency than individual user comfort.

- Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers (For example, network printers)**

OS balances individual performance with controlled access to shared network resources.

Handheld computers such as smartphones and tablets, optimized for usability and battery life

OS - user experience and energy efficiency are more important than raw performance.

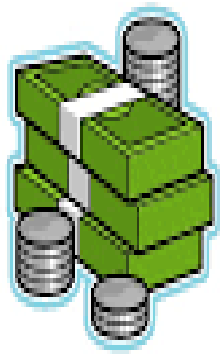
Some computers have little or no user interface, such as embedded computers in devices and automobiles.

The OS works silently in the background, focusing on correctness and reliability.

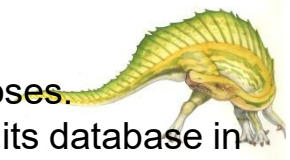
The user view of an operating system varies with the type of system. Shared systems emphasize fair resource utilization, dedicated systems focus on individual performance, handheld systems optimize usability and battery life, and embedded systems prioritize reliability with minimal user interaction.

ICT and Banks

- Banks use mainframe computers to maintain customer accounts.
- They store a record of each customer's withdrawals and deposits.
- Each bank mainframe is used to operate a network of ATMs.



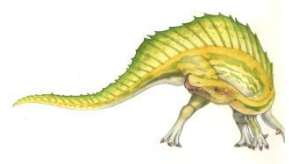
- who uses mainframes? Just about everyone has used a mainframe computer at one point or another. If you ever used an automated teller machine (ATM) to interact with your bank account, you used a mainframe.
- The mainframe is often used by IT organizations to host the most important, mission-critical applications. These applications typically include customer order processing, financial transactions, production and inventory control, payroll, as well as many other types of work.
- Both e-business and e-commerce use mainframe computers to perform business functions and exchange money over the Internet. Banking institutions, stock brokerage firms, insurance agencies and Fortune 500 companies are just a few examples of public and private sectors that maintain information and transfer data via mainframes. Whether a business processes millions of customer orders, performs financial transactions, pays employees or tracks production and inventory, a mainframe computer is the only machine that has the storage, speed and capacity to run successful e-commerce and e-business activities.
- mainframes are used basically for transaction purposes. Mainframes might need to do millions of updates to its database in a second when it is being used by retailers, airlines, banks etc.



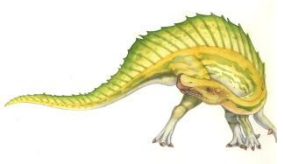
System View

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer. Also, it is concerned with the operation and control of I/O devices.

From the system's perspective, the OS ensures that hardware resources are used efficiently and safely.



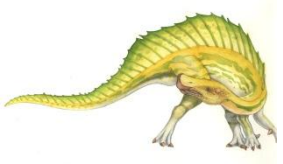
Feature	User View	System View
Focus	User interaction and experience	Hardware and resource management
Interface	GUI, CLI	Kernel, system calls
Primary Concern	Usability and accessibility	Efficiency and optimization
Security	Passwords, Permissions	Process isolation, access control
Example OS Components	Desktop UI, File Manager	Kernel, Process Scheduler
Users	End-users, application users	OS developers, system admins



- “The one program running at all times on the computer” is the **kernel**.
- Everything else is either
  - a system program (ships with the operating system) , or
  - an application program.

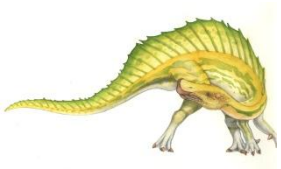
The kernel is a computer program that is the core of a computer's operating system, with complete control over everything in the system.

On most systems, it is one of the first programs loaded on start-up (after the bootloader). It handles the rest of start-up as well as input/output requests from software, translating them into data-processing instructions for the central processing unit. It handles memory and peripherals like keyboards, monitors, printers, and speakers.



Computer Startup

- **bootstrap program** is loaded at power-up or reboot
  - Typically stored in ROM or EEPROM(It can be changed but not frequently. Eg., factory installed pgms in smartphones), generally known as **firmware**
  - Initializes all aspects of system (From CPU registers to device controllers to memory contents)
  - It locates and loads operating system kernel into memory and starts execution
  - On UNIX, the first system process is “init” and it starts many other daemons.
  - Once this step is done, system waits for some events to occur.
  - Interrupt: Occurence of an event. Two types of interrupt: Hardware interrupt and Software interrupt
  - Hardware interrupt: Hardware device can trigger interrupt to CPU at any time by sending signal through system bus.
  - Software interrupt: Program may interrupt by executing a special operation called a **system call** or monitor call.



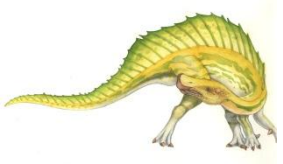
System calls provide the means for a user program to ask the operating system to perform tasks reserved for the operating system on the user program's behalf.

System call instructions:

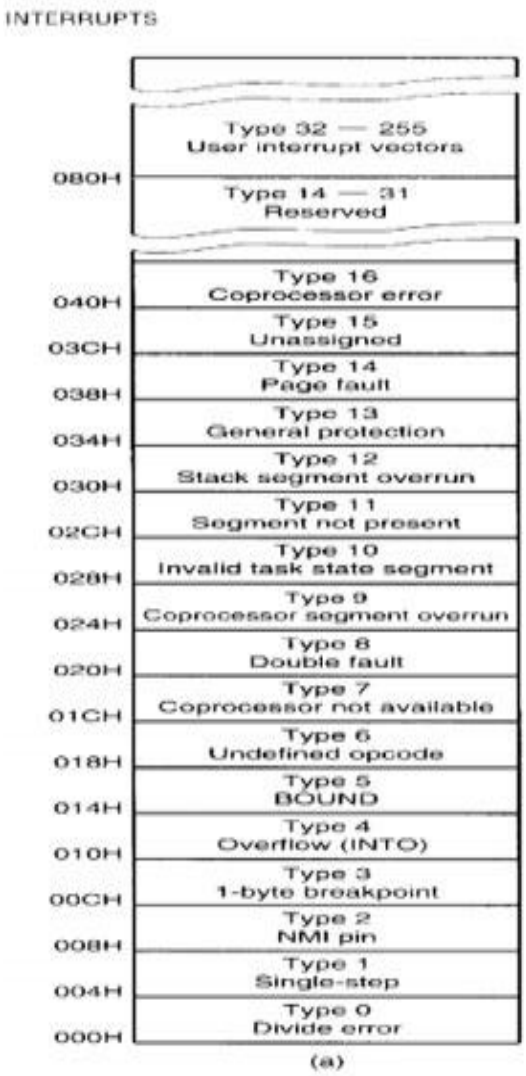
trap and syscall

Whenever user application executes illegal instruction or tries to access invalid memory address hardware traps it to OS. An appropriate error message is given, and the memory of the program may be dumped.

The memory dump is usually written to a file so that the user or programmer can examine it and perhaps correct it and restart the program



What happens when CPU is interrupted?



CPU stops what it is doing and immediately transfers execution to a fixed location. The fixed location usually contains the starting address where the service routine for the interrupt is located. The interrupt service routine executes; on completion, the CPU resumes the interrupted computation.

Whenever an interrupt occurs, general interrupt handler will inspect the interrupt and transfer control to a specific interrupt handler.

To handle interrupts quickly, for every device, there will be an interrupt number which will index the interrupt vector table. At a particular index, the starting address of interupt service routine of a particular device is placed.

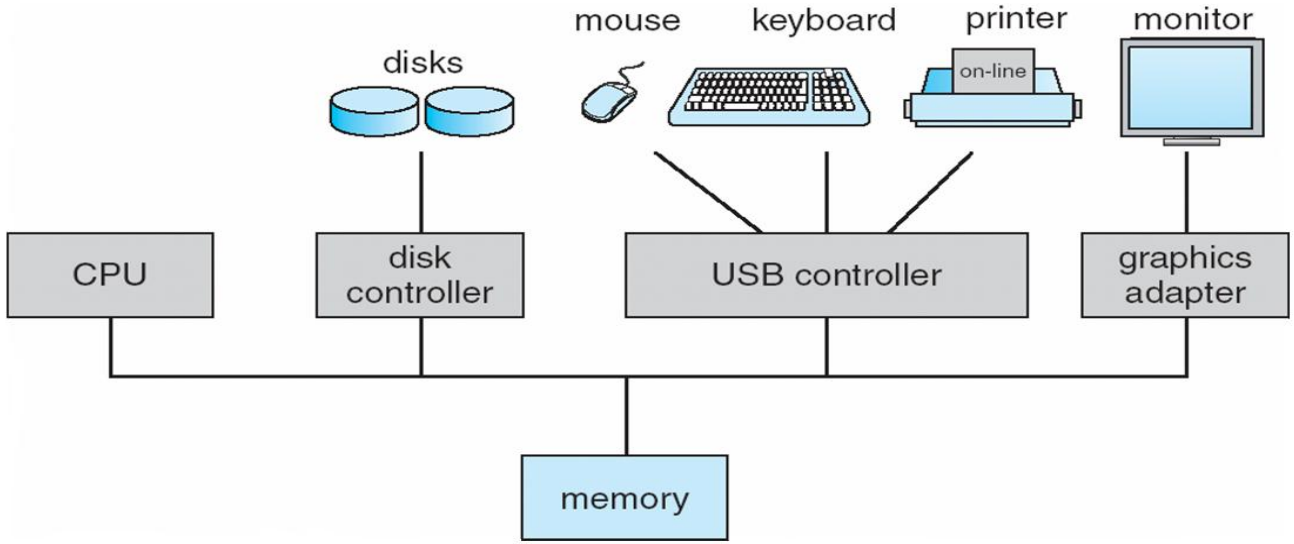
On PCs, the interrupt vector table consists of 256 4-byte pointers, and resides in the first 1 K of addressable memory. Each interrupt number is reserved for a specific purpose.

Computer System Organization

- Computer-system operation

One or more CPUs, device controllers connect through common bus providing access to shared memory.

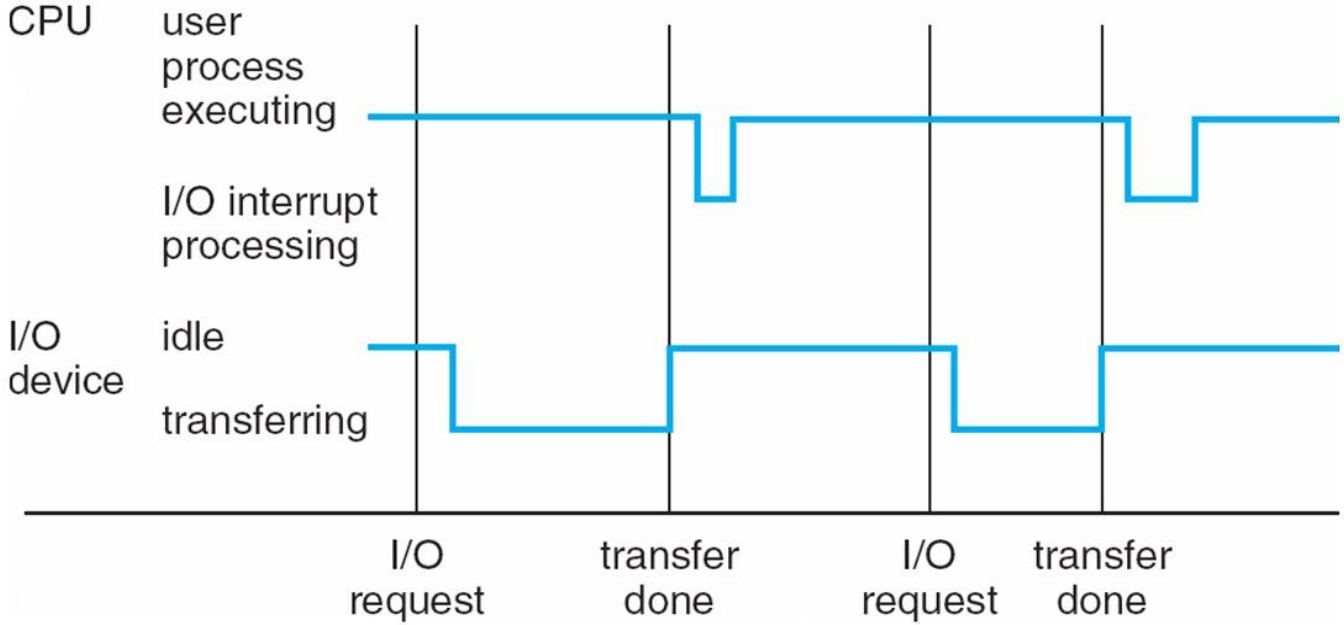
- Concurrent execution of CPUs and devices competing for memory cycles. To ensure orderly access to the shared memory, a memory controller synchronizes access to the memory.

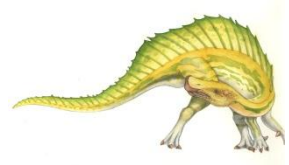


Common Functions of Interrupts

- Interrupt architecture must save the address of the interrupted instruction. The operating system preserves the state of the CPU by storing registers and the program counter
- A trap or exception is a software-generated interrupt caused either by an error or a user request
- An operating system is interrupt driven

Interrupt Timeline





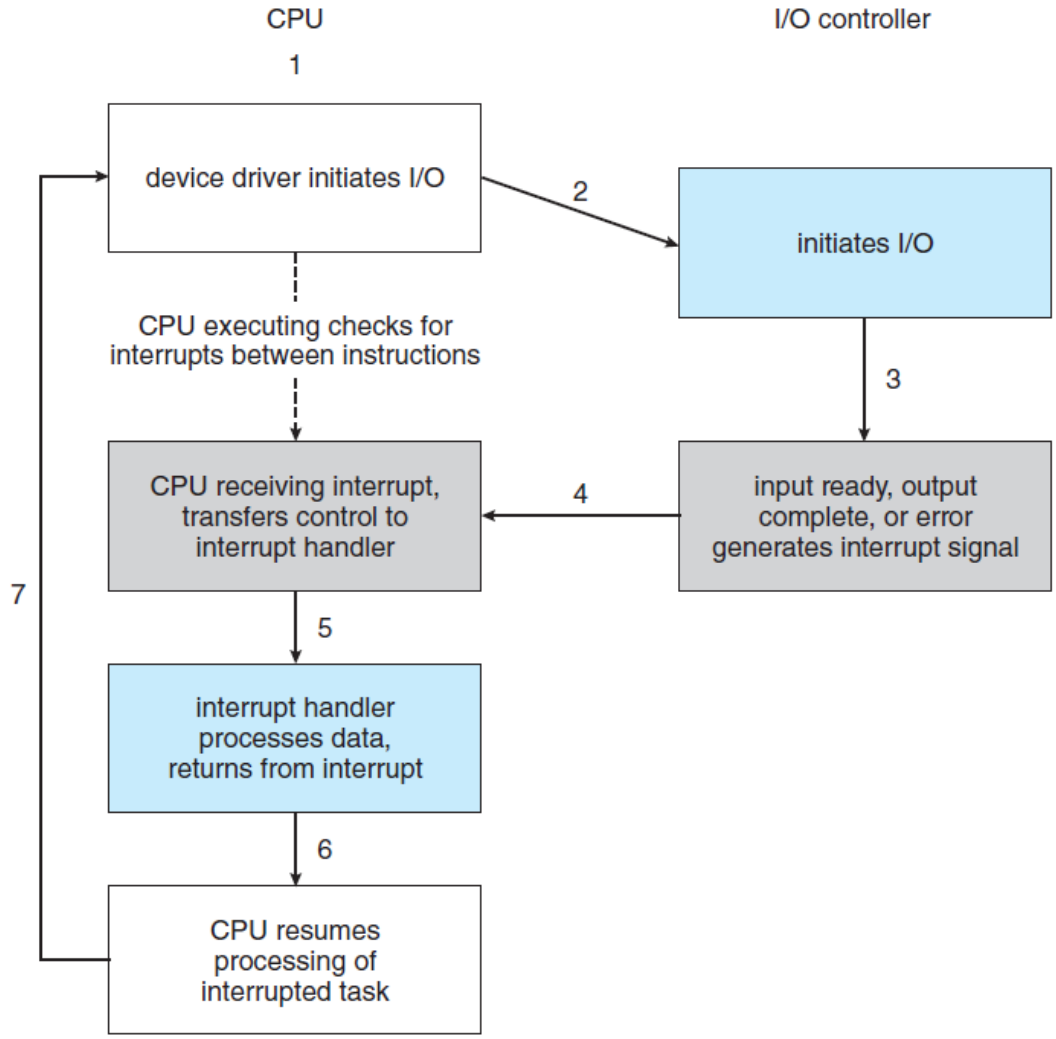
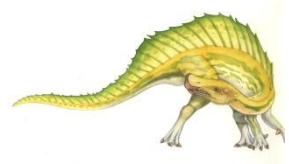


Figure 1.4 Interrupt-driven I/O cycle.



vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Figure 1.5 Intel processor event-vector table.

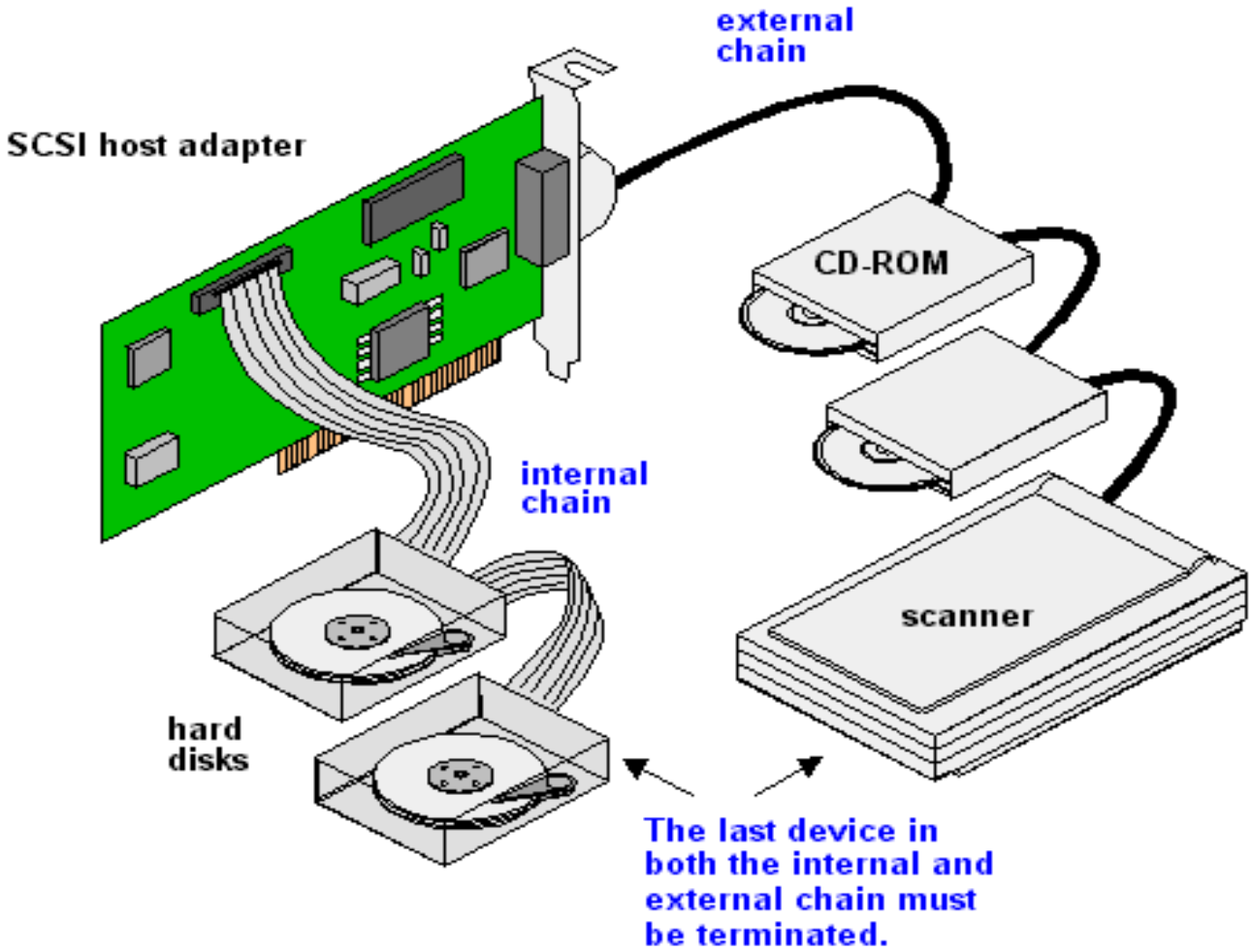
I/O Structure

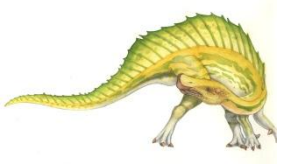
Device controller:

Each device controller is in charge of a specific type of device. Depending on the controller, more than one device may be attached. For instance, seven or more devices can be attached to the small computer-systems interface (SCSI) controller

A device controller maintains some local buffer storage and a set of special-purpose registers. The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.

Operating systems have a device driver for each device controller. This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device. **A large portion of operating system code is dedicated to managing I/O** , both because of its importance to the reliability and performance of a system and because of the varying nature of the devices.


external
chain
SCSI host adapter
CD-ROM
internal
chain
scanner
hard
disks
The last device in
both the internal and
external chain must
be terminated.

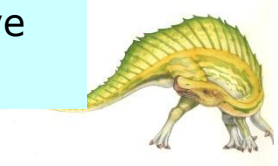


Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.
A **kilobyte**, or **KB**, is 1,024 bytes
a **megabyte**, or **MB**, is $1,024^2$ bytes
a **gigabyte**, or **GB**, is $1,024^3$ bytes
a **terabyte**, or **TB**, is $1,024^4$ bytes
a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).



Storage Structure

- Main memory (RAM) – only large storage media that the CPU can access directly (DRAM)
  - **Random access**
  - Typically **volatile**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- Hard disks – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
- **Solid-state disks** – faster than hard disks, nonvolatile
  - Various technologies
  - Becoming more popular

Load instruction: It loads a byte or a word from RAM to CPU register

Store instruction: It moves data from CPU register to RAM

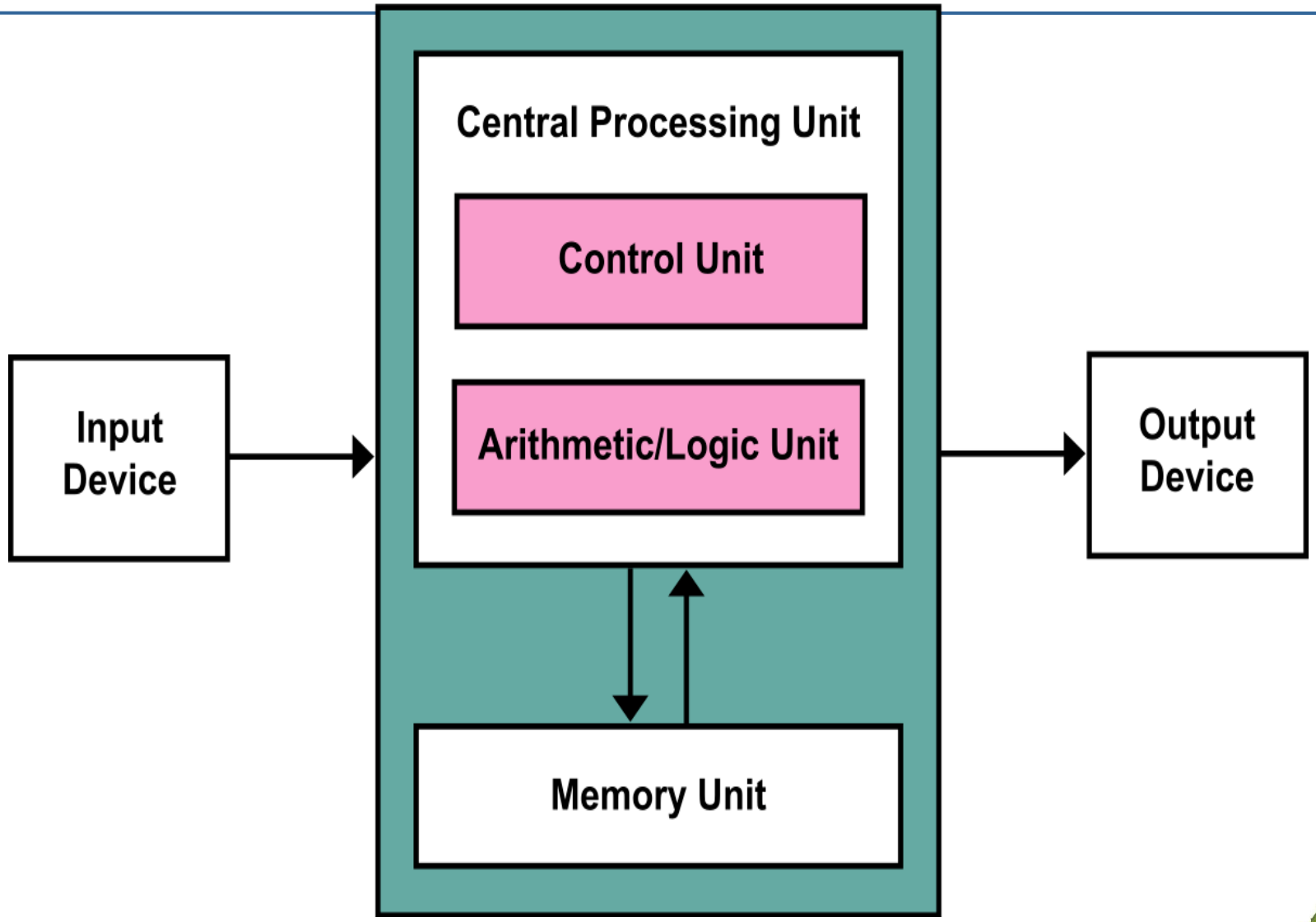
Von Neumann architecture: Von Neumann Architecture is a computer architecture proposed by John von Neumann in which program instructions and data are stored in the same memory and share the same bus.

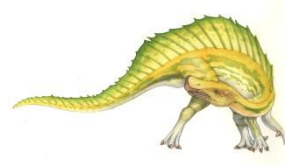
Instruction-execution cycle:

1) Fetches an instruction from memory and stores that instruction in the instruction register.

2) The instruction is then decoded and may cause operands to be fetched from memory and stored in some internal register.

3) After the instruction on the operands has been executed, the result may be stored back in memory.


Central Processing Unit
Control Unit
Arithmetic/Logic Unit
Input Device
Output Device
Memory Unit



Storage Hierarchy

- Storage systems organized in hierarchy (The main differences among the various storage system lie in)
  - Speed
  - Cost
  - Volatility
- Caching – copying information into faster storage system; main memory can be viewed as a cache for secondary storage

Storage-Device Hierarchy

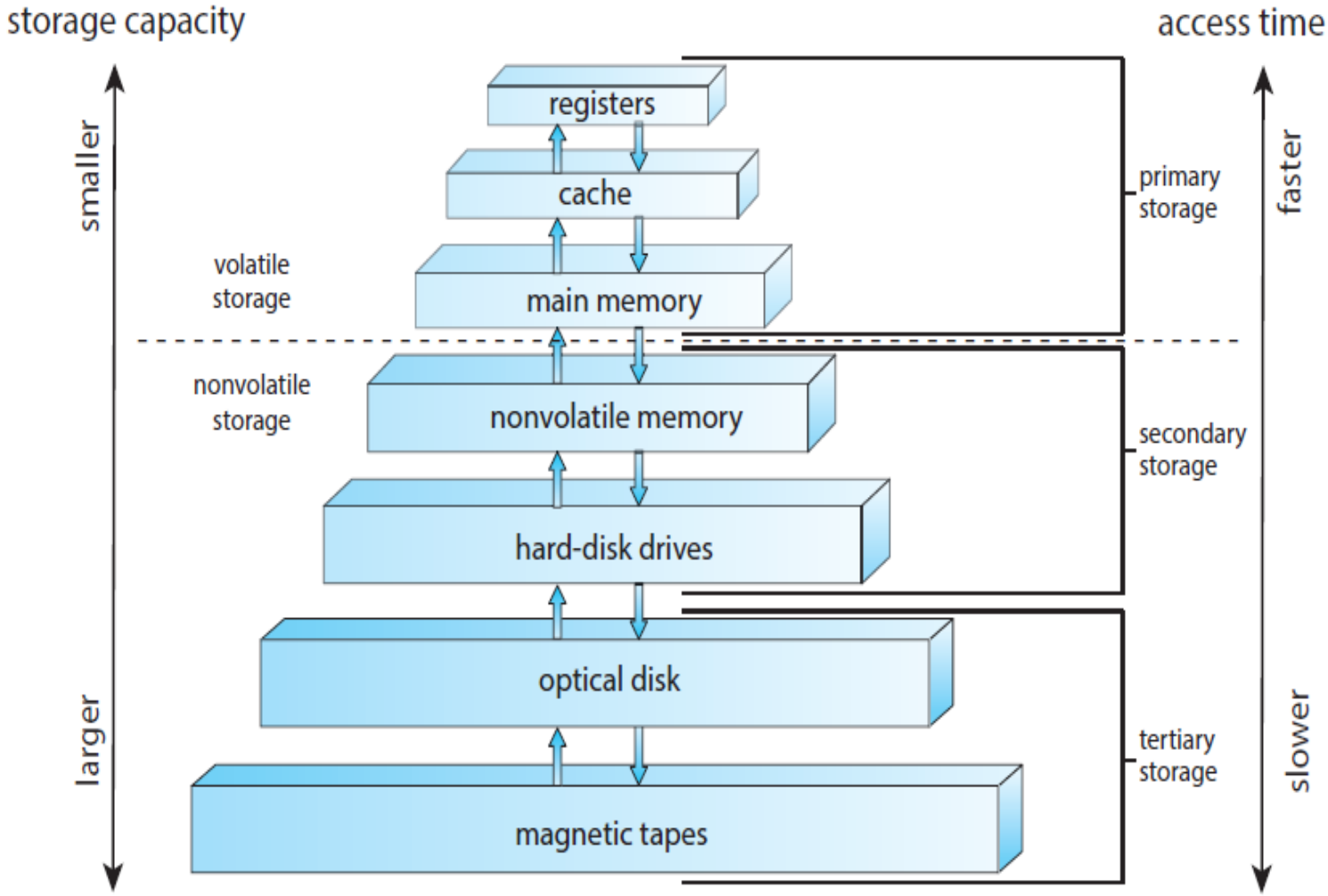
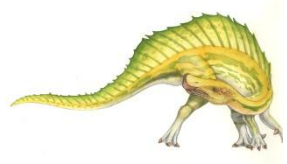


Figure 1.6 Storage-device hierarchy.

Solid state disk: A solid-state drive (SSD) is a nonvolatile storage device that stores **persistent data on solid-state flash memory.** Solid-state drives actually aren't hard drives in the traditional sense of the term, as there are no moving parts involved.

An SSD, on the other hand, has an **array of semiconductor memory** organized as a disk drive, **using integrated circuits (ICs)** rather than magnetic or optical storage media.

SSDs have much **lower random access and read access latency than HDDs,** making them ideal for both heavy read and random workloads. That **lower latency is due to the ability of flash SSD** to read data directly and immediately from a specific flash SSD cell location. High-performance servers, laptops, desktops or any application that needs to deliver information in real-time or near real-time can benefit from solid-state drive technology.



Two variants of solid state disks:

1) It stores data in a large DRAM array during normal operation but also contains a hidden magnetic hard disk and a battery for backup power. If external power is interrupted, this solid-state disk's controller copies the data from RAM to the magnetic disk. When external power is restored, the controller copies the data back into RAM .

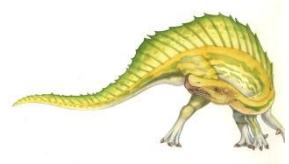
2) Based on flash memory which is slower than DRAM but needs no power to retain its contents. It contains no moving part.

NVRAM: It is DRAM with battery backup power. This memory can be as fast as DRAM and (as long as the battery lasts) is nonvolatile.

ArxCis-NV DIMM

Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there
- Cache smaller than storage being cached
  - Cache management important design problem
  - Cache size and replacement policy



Direct Memory Access Structure

Between CPU and I/O devices can be processed by several modes in the computer systems. There are two main ways that are used in I/O operations, and they include programmed I/O and interrupt initiated I/O.

Programmed I/O: It refers to data transfers initiated by a CPU under driver software control to access registers or memory on a device.

Programmed I/O basically works in the following steps:

1) CPU requests I/O operation
2) I/O module performs operation
3) I/O module sets status bits
4) CPU checks status bits periodically
5) I/O module does not inform CPU directly
6) I/O module does not interrupt CPU
7) CPU may wait or come back later

In this case, the I/O device does not have direct access to the memory unit. A transfer from I/O device to memory requires the execution of several instructions by the CPU. The CPU, while waiting, must repeatedly check the status of the I/O module, and this process is known as Polling. As a result, the level of the performance of the entire system is severely degraded

Interrupt driven I/O:

The CPU issues commands to the I/O module then proceeds with its normal work until interrupted by I/O device on completion of its work.

Below are the basic operations of Interrupt driven I/O

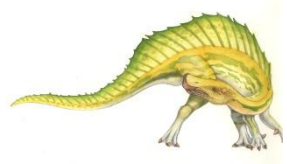
CPU issues read command

I/O module gets data from peripheral whilst CPU does other work

I/O module interrupts CPU

CPU requests data

I/O module transfers data



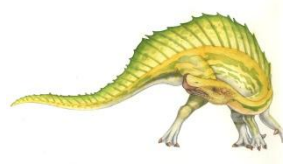
Direct Memory Access:

It means CPU grants I/O module authority to read from or write to memory without involvement.

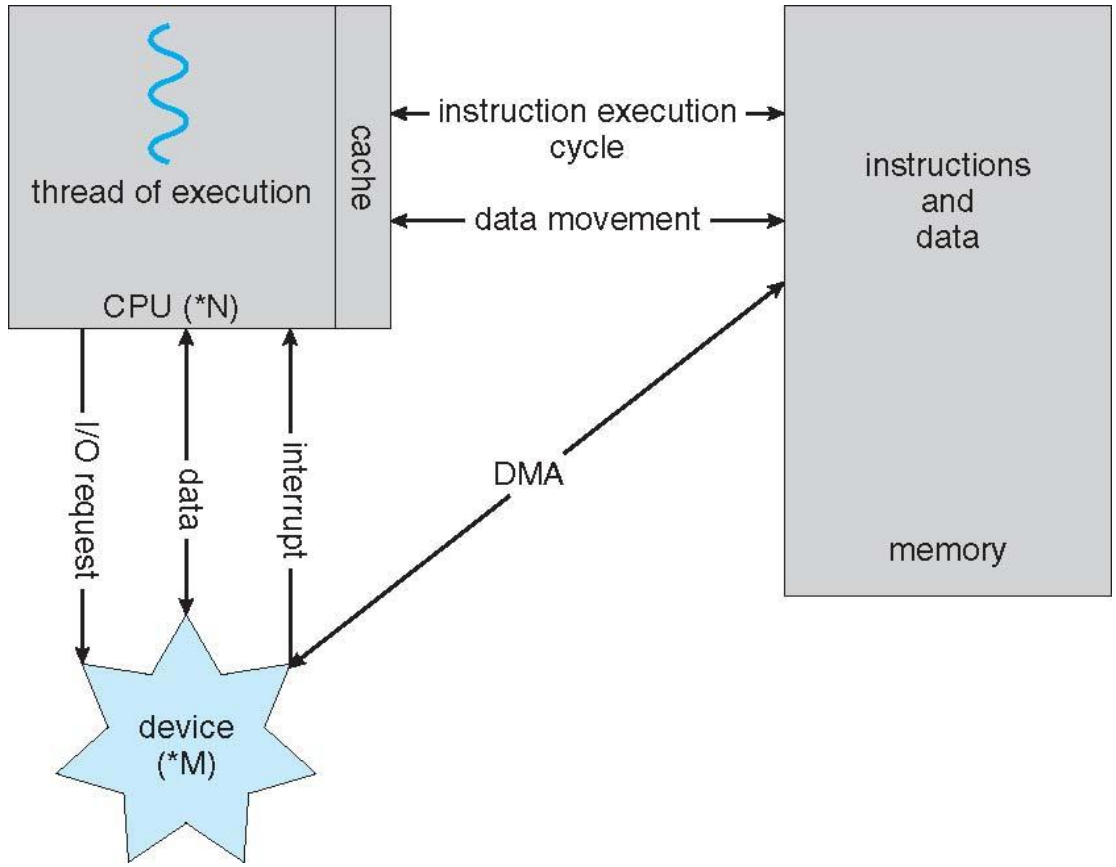
DMA module controls exchange of data between main memory and the I/O device.

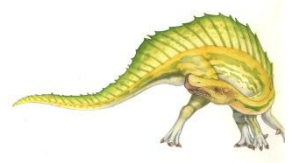
Because of DMA device can transfer data directly to and from memory, rather than using the CPU as an intermediary, and can thus relieve congestion on the bus. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus.



How a Modern Computer Works





Computer-System Architecture

A computer system can be organized in a number of different ways, which we can categorize roughly according to the number of general-purpose processors used.

1) Single-processor system
2) Multi-processor system
3) Clustered system (a type of multi-processor system)

Single-processor system

On a single- processor system, there is one main CPU capable of executing a general-purpose instruction set, including instructions from user processes.

Almost all single-processor systems have other special-purpose processors as well. They may come in the form of device-specific processors, such as disk, keyboard, and graphics controllers;

Example: A hard disk-controller microprocessor receives a sequence of requests from the main CPU and implements its own disk queue and scheduling algorithm. This arrangement relieves the main CPU of the overhead of disk scheduling.

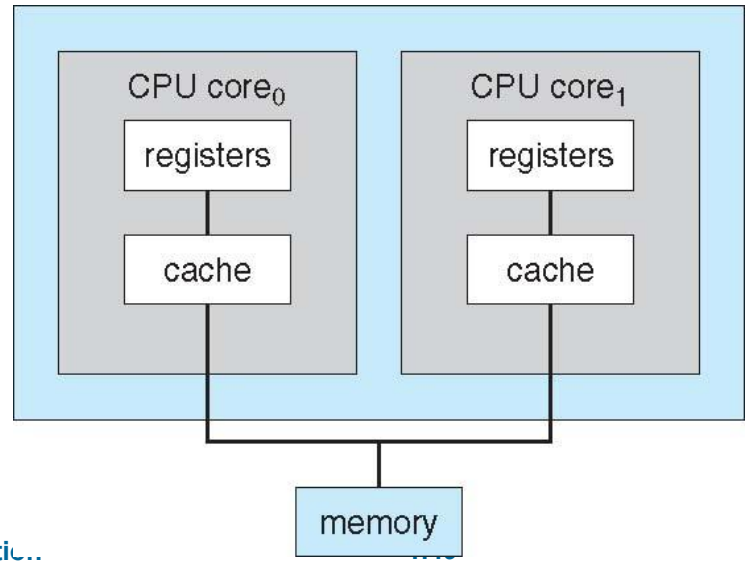
Multi-core system

Single Processor chips with multiple processing units

Cores

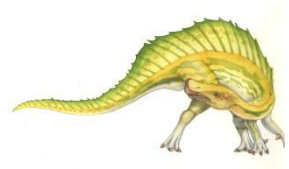
Cores –ALU, registers

Share resources like cache and main memory



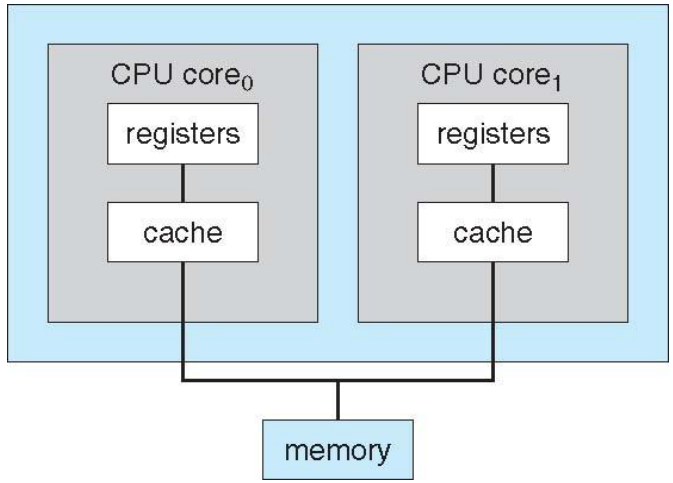
Multi-core

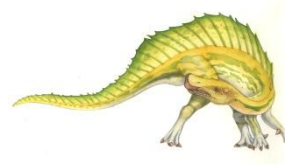
Intel Core i7 with 8 cores



A Dual-Core Design

A recent trend in CPU design is to include multiple computing cores on a single chip. Such multiprocessor systems are termed multicore. They can be more efficient than multiple chips with single cores because on-chip communication is faster than between-chip communication. In addition, one chip with multiple cores uses significantly less power than multiple single-core chips.



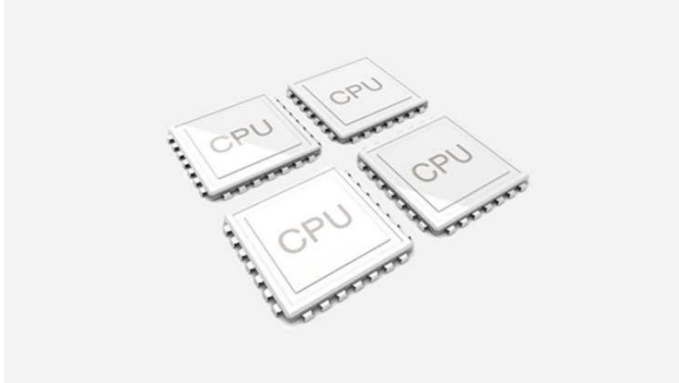


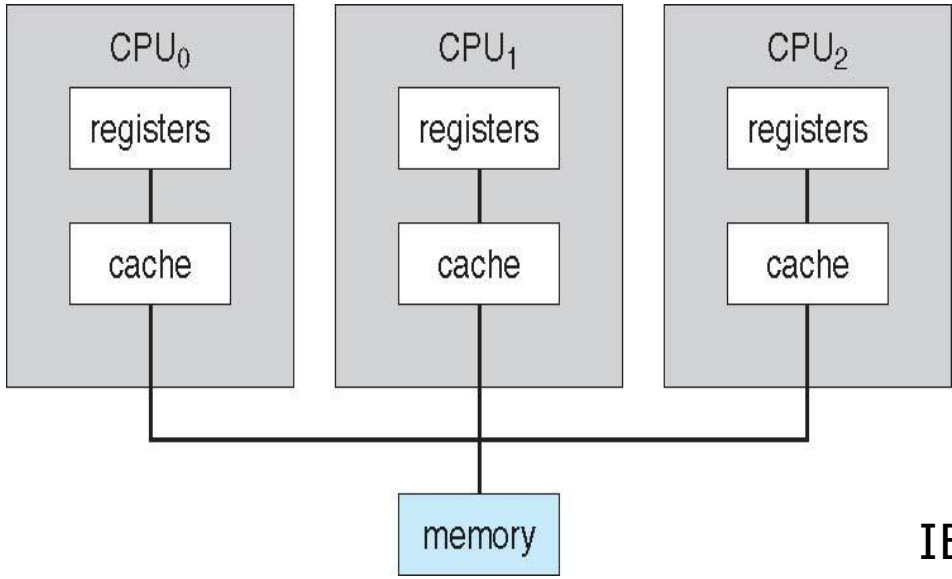
Multi-processor system

Two (or more) processors, each with a single-core CPU

They have two or more processors in close communication, **sharing the computer bus and memory, and peripheral devices.**

Multiple processors may be separate chips or multiple *cores* on the same chip





IBM powersystem E980

Multiple processor

Advantages of multiprocessor systems:

1) **Increased throughput**: By increasing the number of processors, we expect to get more work done in less time.

Does the speed-up ratio with N processors is N? (When multiple processors cooperate on a task, a certain amount of overhead is incurred in keeping all the parts working correctly. This overhead, plus contention for shared resources, lowers the expected gain from additional processors.)

2) **Economy of scale**: Multiprocessor systems can cost less than equivalent multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.

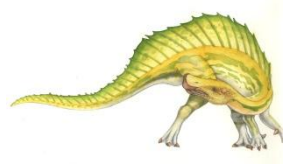
3) **Increased reliability**: If functions can be distributed properly among several processors, then the failure of one processor will not halt the system, only slow it down.

How “increased reliability” is achieved?

Graceful degradation is the ability of a computer, machine, electronic system or network to maintain limited functionality even when a large portion of it has been destroyed or rendered inoperative.

The purpose of graceful degradation is to prevent catastrophic failure.

Ideally, even the simultaneous loss of multiple components does not cause downtime in a system with this feature. In graceful degradation, the operating efficiency or speed declines **gradually** as an increasing number of components fail.



Fault tolerance requires a mechanism to allow the failure to be detected, diagnosed, and, if possible, corrected.

Tandem Computers, Inc. was the dominant manufacturer of fault-tolerant computer systems for ATM networks, banks, stock exchanges, telephone switching centers etc.

The HP NonStop (a series of server computers)(formerly Tandem) system uses both hardware and software duplication to ensure continued operation despite faults.

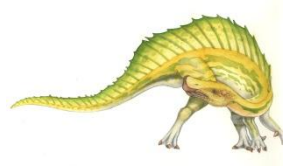
The system consists of multiple pairs of CPU s, working in “lockstep”.

Both processors in the pair execute each instruction and compare the results.

If the results differ, then one CPU of the pair is at fault, and both are halted.

The process that was being executed is then moved to another pair of CPU s, and the instruction that failed is restarted.

This solution is expensive, since it involves special hardware and considerable hardware duplication.



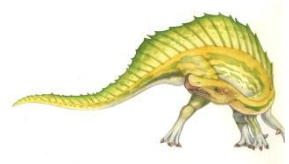
2 types of multi-processor systems:

1) Asymmetric multiprocessing

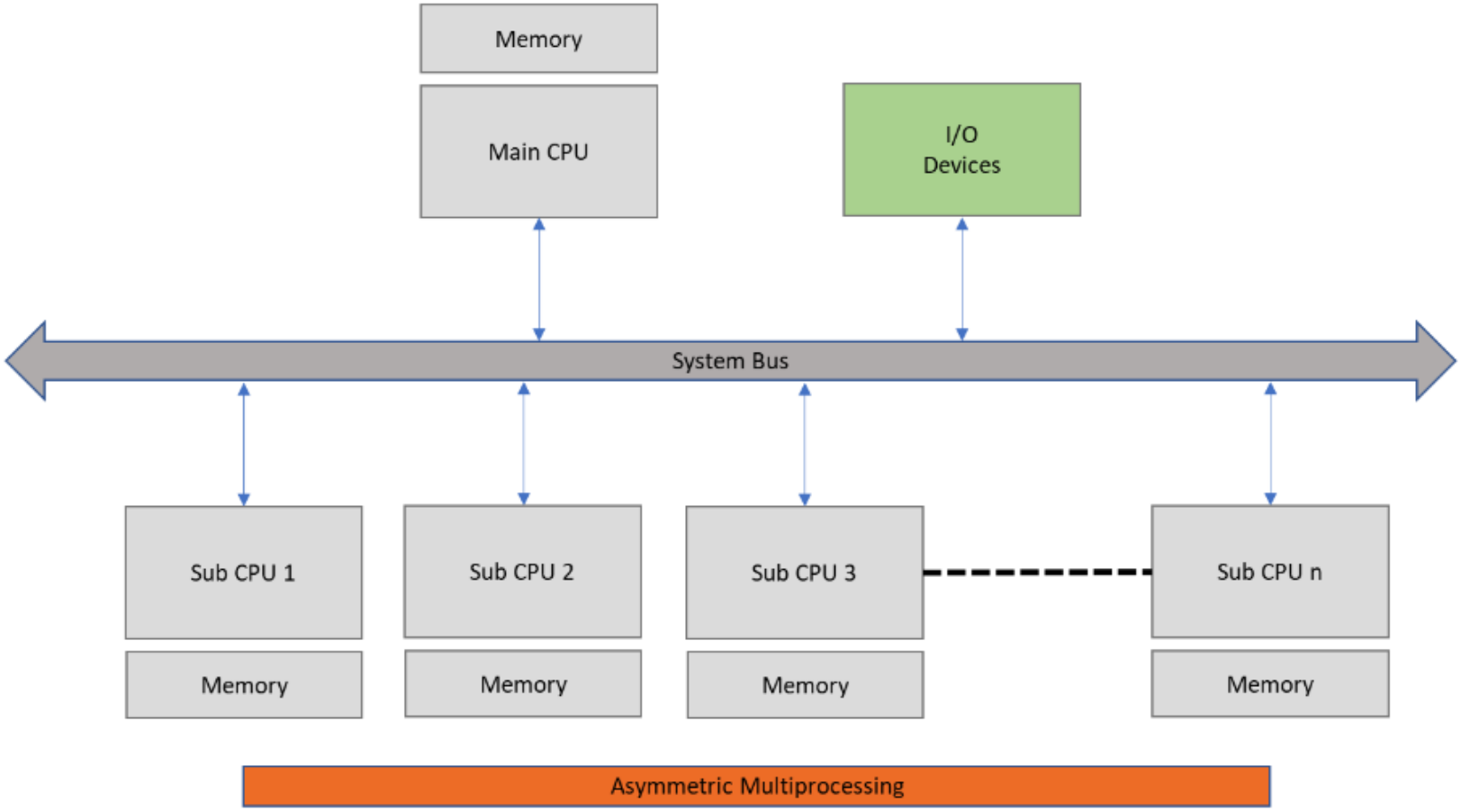
2) Symmetric multiprocessing

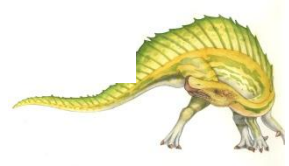
Asymmetric multiprocessing (Separate Memory)

Each processor is assigned a specific task. A boss processor controls the system; the other processors either look to the boss for instruction or have predefined tasks. This scheme defines a **boss–worker relationship**. The boss processor schedules and allocates work to the worker processors.



Asymmetric multiprocessing



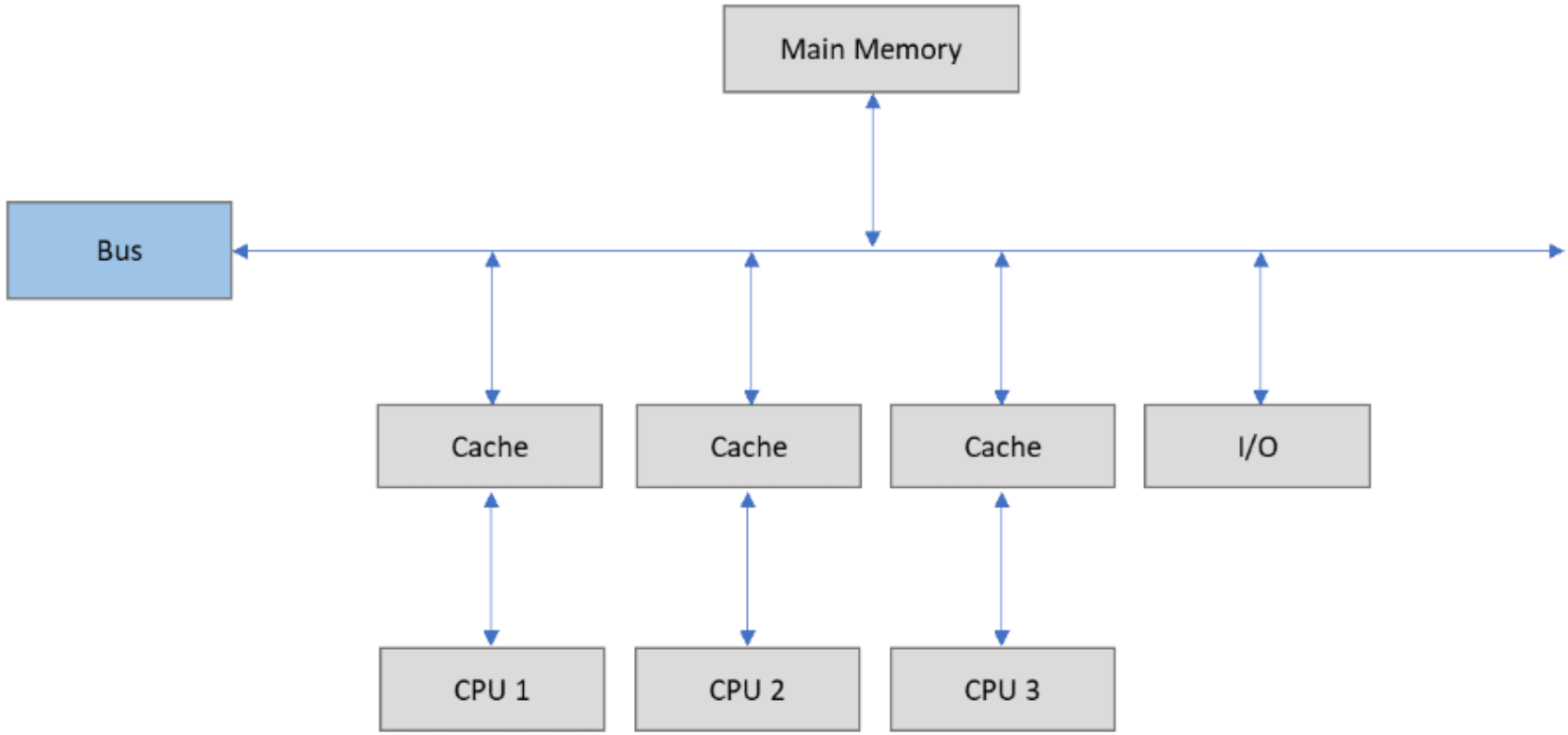


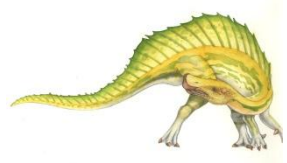
Symmetric multiprocessing:

Each processor performs all tasks. It involves a multiprocessor computer hardware and software architecture where two or more identical processors are connected to a single, shared main memory, have full access to all input and output devices, and are controlled by a single operating system instance that treats all processors equally, reserving none for special purposes.

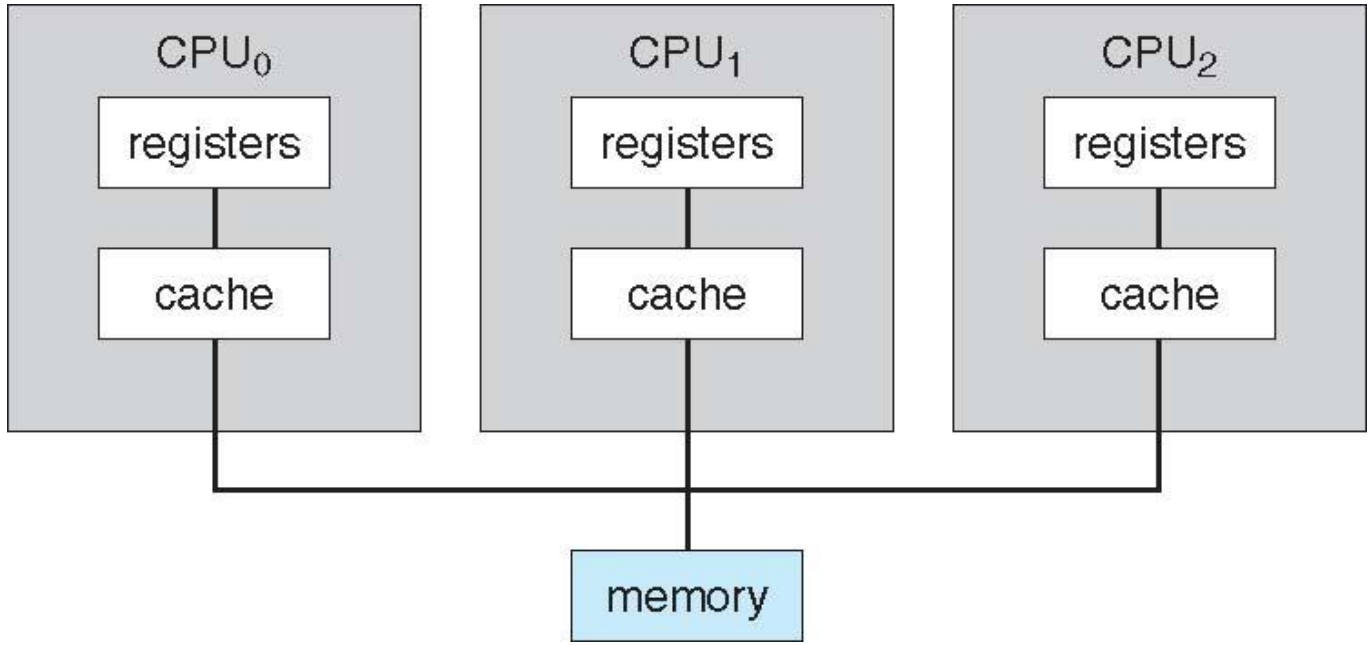
There's just one kernel in SMP mode, and it's run by all cores. In AMP mode, each core has its own copy of a kernel, which could be different (heterogeneous operating systems) from, or identical (homogenous operating systems) to, the one the other core is executing.

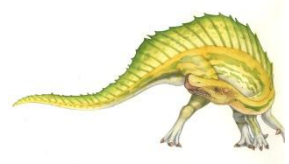
Symmetric multiprocessing





Symmetric Multiprocessing Architecture

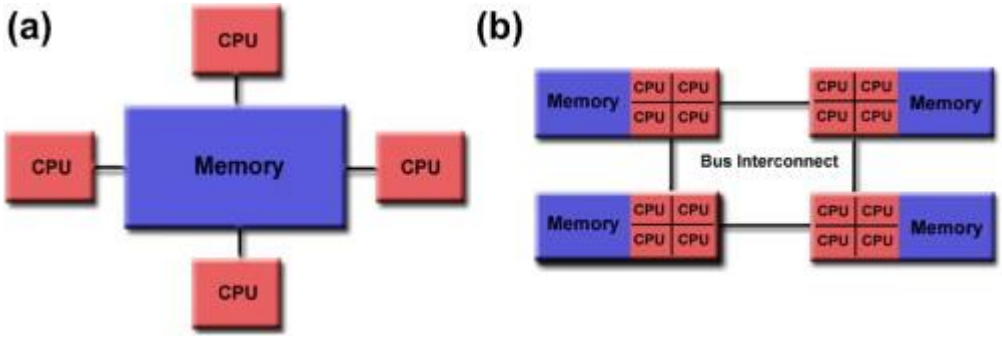




UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) are two different methods to manage memory in multi-processor systems. They define how processors access system memory and impact performance.

UMA(Uniform Memory Access) : It is defined as the situation in which access to any RAM from any CPU takes the same amount of time. **all CPUs share the same memory**.

NUMA(Non-Uniform Memory Access): Some parts of memory may take longer to access than other parts.



Basic shared memory architecture. (a) Uniform memory access (UMA) architecture. (b) Non-uniform memory access (NUMA) architecture.

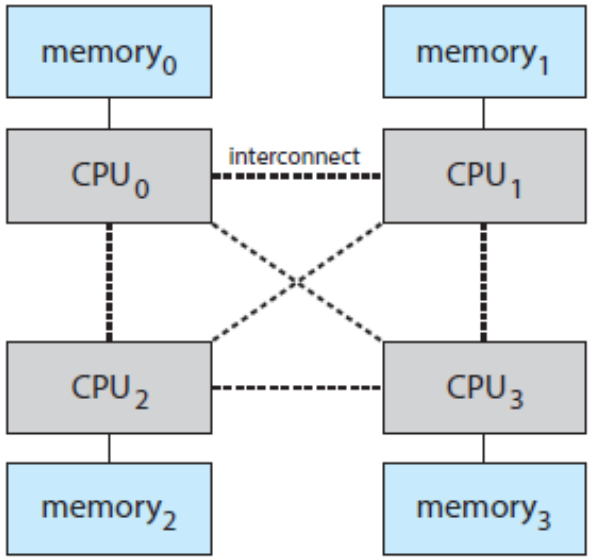
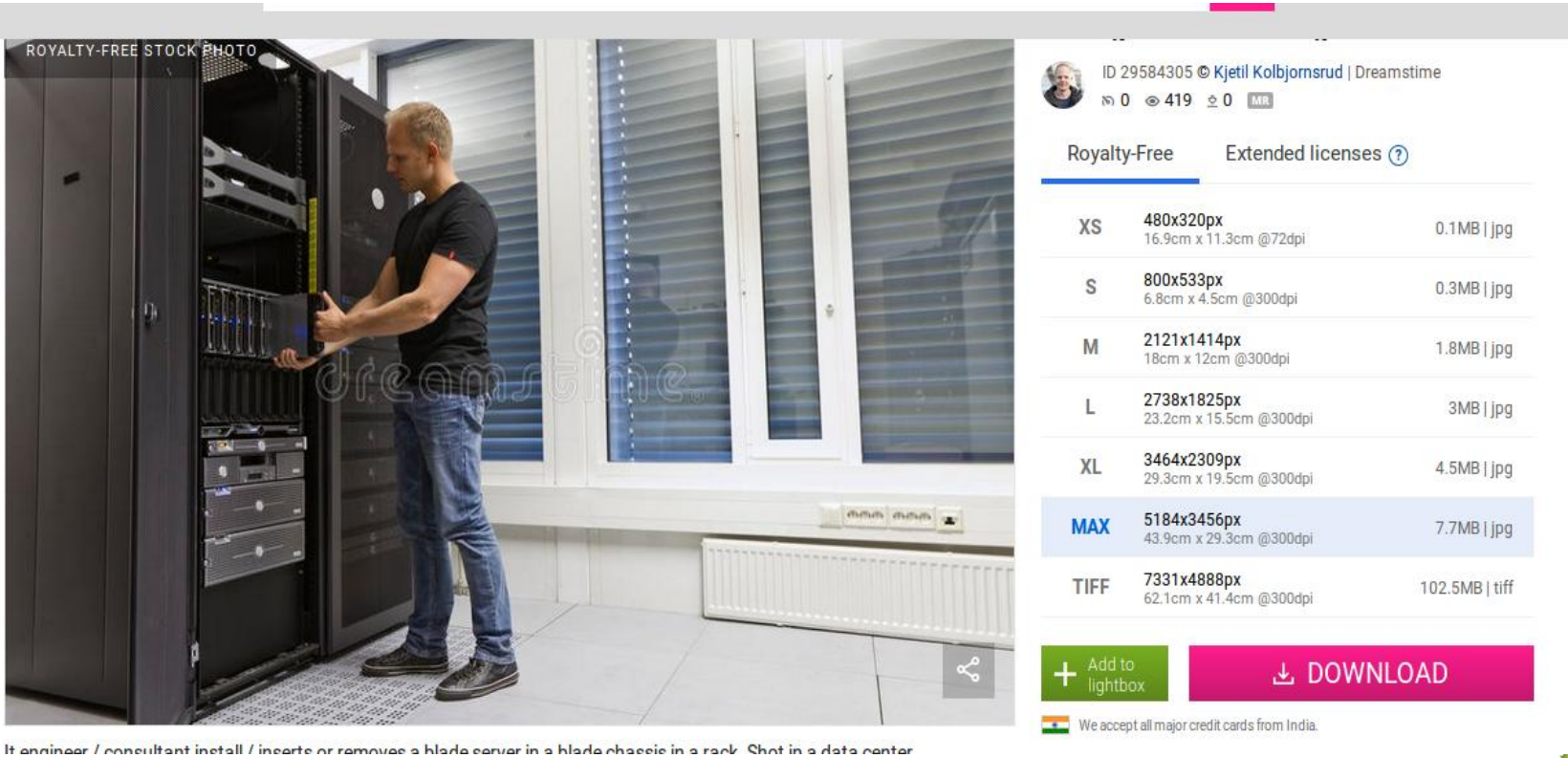


Figure 1.10 NUMA multiprocessing architecture.

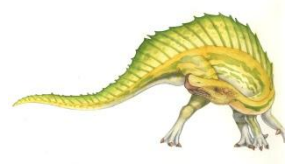
Blade servers: Multiple processor boards, I/O boards, and networking boards are placed in the same chassis. each blade-processor board boots independently and runs its own operating system. These servers consist of multiple independent multiprocessor systems.



Clustered Systems

- Like multiprocessor systems, but multiple systems working together
  - Usually sharing storage via a **storage-area network (SAN)**
  - Provides a **high-availability** service which survives failures
    - **Asymmetric clustering** has one machine in hot-standby mode
    - **Symmetric clustering** has multiple nodes running applications, monitoring each other

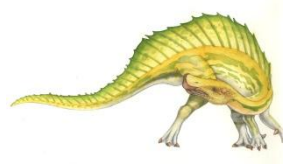
Some clusters are for **high-performance computing (HPC)** (High-performance computing (HPC))



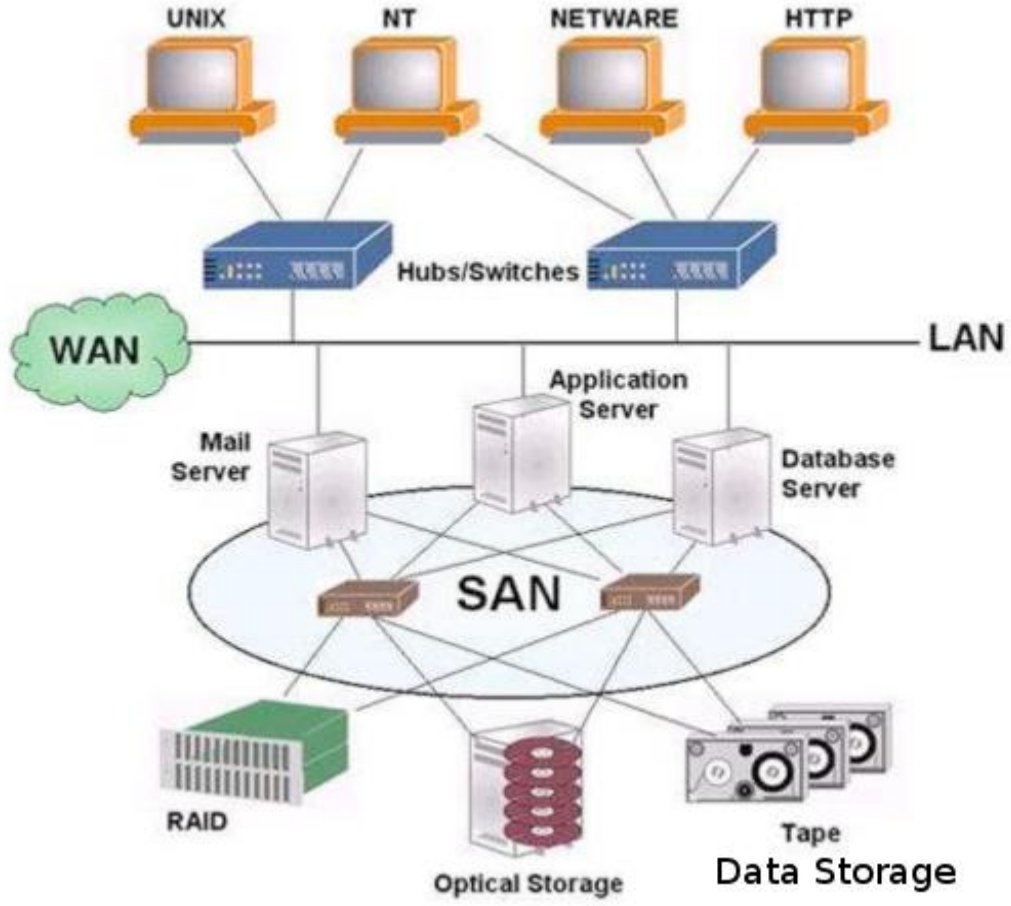
Operating systems use lock managers to organise and serialise the access to resources.

A distributed lock manager (DLM) runs in every machine in a cluster, with an identical copy of a cluster-wide lock database. In this way a DLM provides software applications which are distributed across a cluster on multiple machines with a means to synchronize their accesses to shared resources.

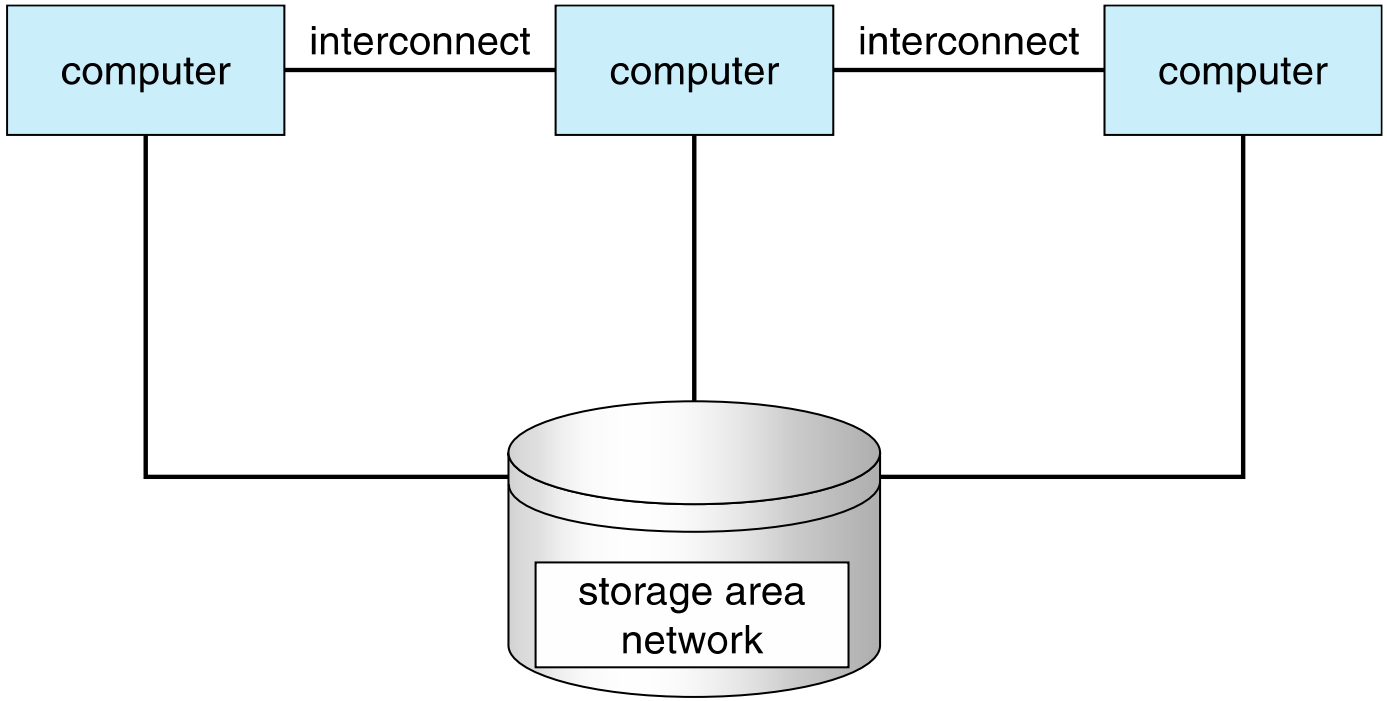
Oracle RAC allows multiple computers to run Oracle RDBMS software simultaneously while accessing a single database, thus providing clustering.



A storage-area network (SAN) is a dedicated high-speed network that interconnects and presents shared pools of storage devices to multiple servers.



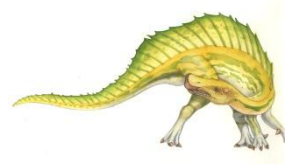
Clustered Systems



Operating System Structure

- **Multiprogramming** (**Batch system**) needed for efficiency
  - Single program cannot keep CPU and I/O devices busy at all times
  - So, single user have multiple programs running. Multiprogramming organizes jobs (code and data) so CPU always has one to execute
  - A subset of total jobs in system is kept in memory
  - One job selected and run via **job scheduling**
  - When it has to wait (for I/O for example), OS switches to another job

- **Timesharing** (**multitasking**) is logical extension of multiprogramming in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory **process (A program loaded into memory and executing)**



Click to add Title

Job scheduling: If several jobs are ready to be brought into memory, and if there is not enough room for all of them, then the system must choose among them. Making this decision involves job scheduling.

CPU scheduling: if several jobs are ready to run at the same time, the system must choose from main memory which job will run first. Making this decision is CPU scheduling

Swapping: It is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

Virtual memory: It is a region in harddisk and stores areas(data&code) of RAM that have not been used recently. This frees up space in RAM to load the new application. Because it does this automatically, you don't even know it is happening, and it makes your computer feel like is has unlimited RAM space even though it has only less memory.

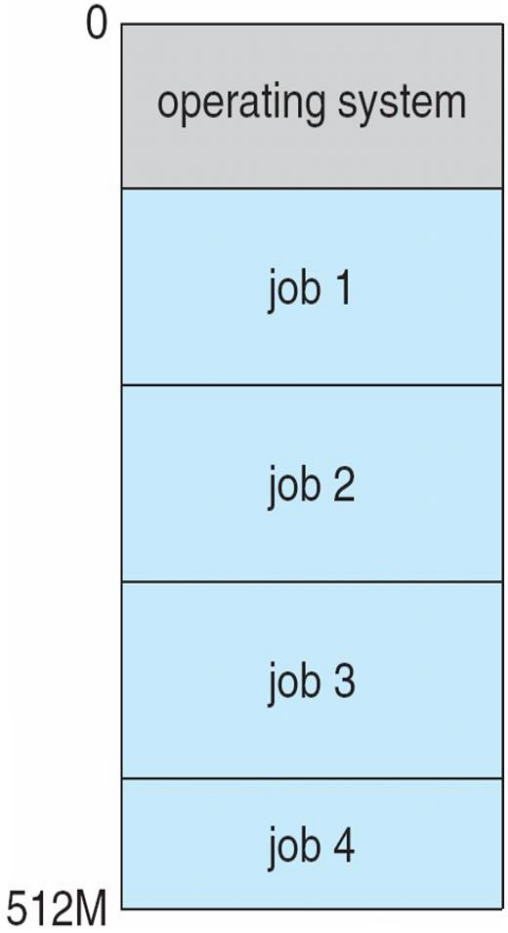
It is technique that allows the execution of a process that is not completely in memory

What is time-shared operating system?

It allows many users to share the computer simultaneously. It uses a timer and scheduling algorithms to cycle processes rapidly through the CPU,giving each user a share of the resources.Processor's time is shared among multiple users simultaneously.

Since each **action or command in a time-shared system tends to be short**, only a little CPU time is needed for each user. As the **system switches** rapidly from one user to the next, each user is given the **impression that the entire computer system is dedicated to his use**, even though it is being shared among many users.

Memory Layout for Multiprogrammed System



The operating system keeps several jobs in memory simultaneously. Since, in general, main memory is too small to accommodate all jobs, the jobs are kept initially on the disk in the job pool.

This pool consists of all processes residing on disk awaiting allocation of main memory.

Operating-System Operations

- **Interrupt driven** (hardware and software)
  - Hardware interrupt by one of the devices
  - Software interrupt (**exception** or **trap):**
    - Software error (e.g., division by zero, accessing invalid memory address)
    - Request for operating system service
    - Other process problems include infinite loop(the process has infinite loop could prevent correct operation of other processes), processes modifying each other or the operating system

A properly designed operating system must ensure that an incorrect (or malicious)program cannot cause other programs to execute incorrectly.

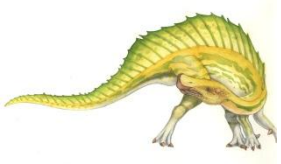
Operating-System Operations (cont.)

- Dual-mode operation allows OS to protect itself and other system components

User mode(1) and kernel mode(0): When CPU is executing user application the system is in user mode. Whenever user application requests OS for some service (read from KB) or trap or interrupt occurs the system will change from user mode to kernel mode. When system boots your HW will be in kernel mode. whenever the operating system gains control of the computer, it is in kernel mode. The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.

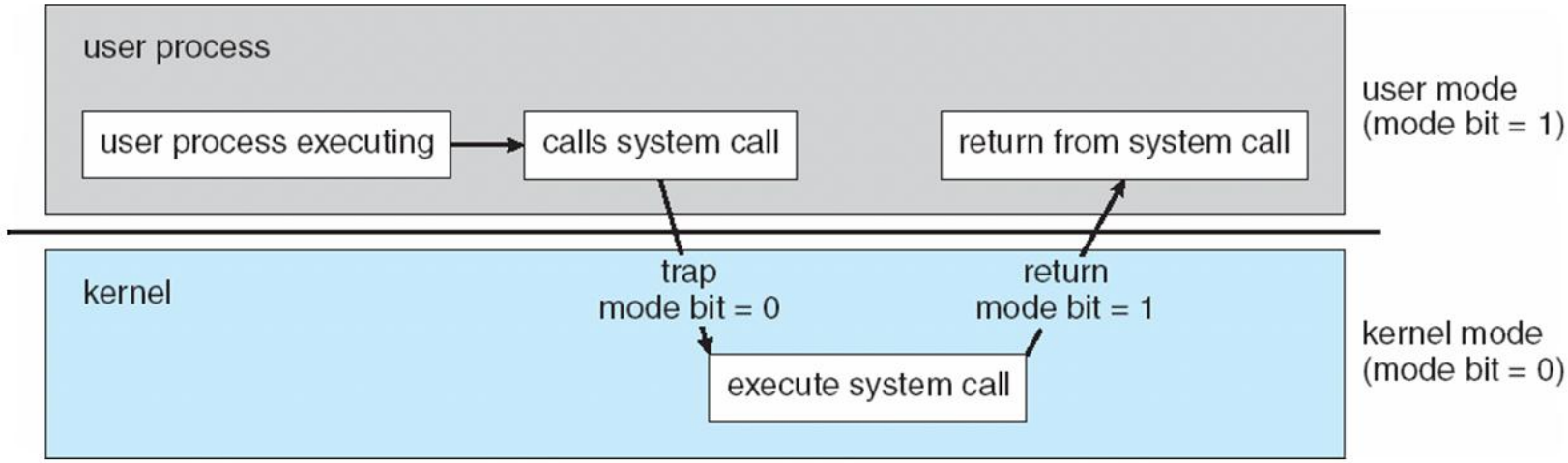
- Mode bit provided by hardware
  - Provides ability to distinguish when system is running user code or kernel code. Some instructions designated as privileged, only executable in kernel mode. Privileged instructions cannot be executed when we are in user mode. Privileged instructions include operations such as I/O and memory management.

Increasingly CPUs support multi-mode operations (CPU's that support virtualization)



Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources
  - Timer is set to interrupt the computer after some time period
  - Keep a counter that is decremented by the physical clock.
  - Operating system set the counter (privileged instruction)
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time



Multimode

What is virtualization?

Virtualization is a technology that allows operating systems to run as applications within other operating systems. Applications are operating system, a server, a storage device or network resources

Virtual machine manager: It manages virtualized resources.

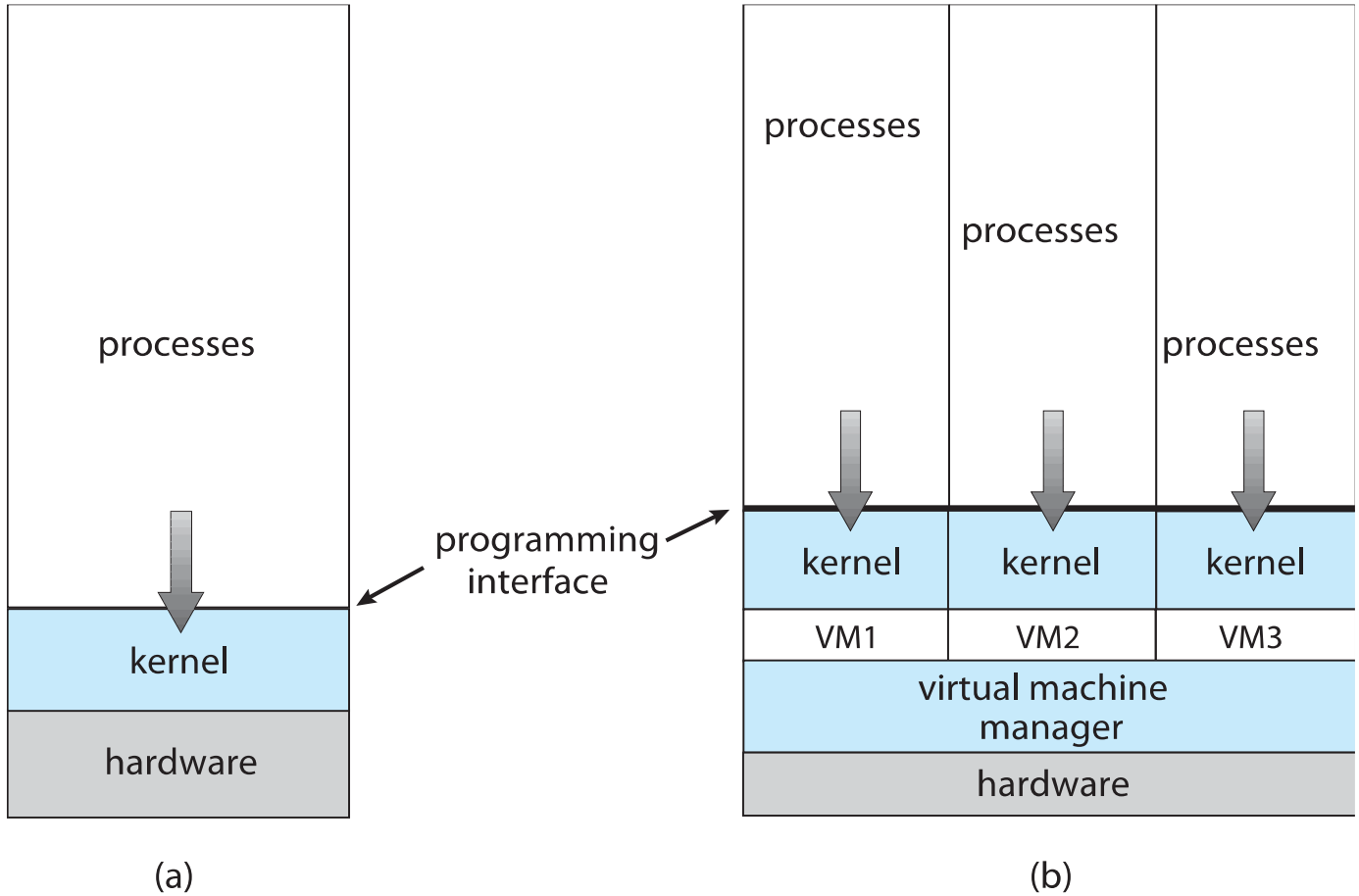
We need a separate mode to indicate the system is in Virtual machine mode.

In this mode, the VMM has more privileges than user processes but fewer than the kernel.

vmware:It allows for multiple virtual machines (VMs) to run on the same physical server. Each VM can run its own operating system (OS), which means multiple OSes can run on one physical server. VMware virtualization lets you run multiple virtual machines on a single physical machine, with each virtual machine sharing the resources of that one physical computer across multiple environments.

Different virtual machines can run different operating systems and multiple applications on the same physical computer.

VMware is an operating system that sits directly on the hardware and is the interface between the hardware and the various operating system.



A computer running (a) a single operating system and (b) three virtual machines.

Timer

As all we know that it is only the operating system that has full control over the CPU, and it is the primary duty of operating system to prevent user programs from getting stuck into an infinite loop or not calling to system services and never returning control back to the operating system.

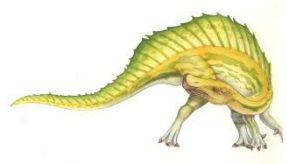
To accomplish all those services an operating system uses a device known as Timer.

The main task of a timer is to interrupt the CPU after a specific period of time. This specific period of time is set by operating system and its value may vary from 1 ms to 1 second.

Variable timer: If the operating system is interested in using variable timer then it is implemented with the help of fixed rate clock or counter and in this case operating system sets the counter.

We can also initialize the value of counter with the amount of time that a process requires to complete its execution.

For example, a program with 7 minutes time limit has its counter initialized to 420,and for every clock tick the counter is decremented by 1. Hence after 420 ticks the control is automatically transferred back to the operating system.



Summary

Power ON ->BIOS / UEFI Boot Loader-> OS Kernel Loaded ->System & User Processes Created ->Ready Queue ->CPU Scheduler ->Process Executes ->Timer Interrupt → Context Switch → Repeat

Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a ***passive entity***, process is an ***active entity***.

- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data

Process termination requires reclaim of any reusable resources. Single-threaded process has one **program counter (It is a register in a processor that has the address of the next instruction to be executed)**

- Multi-threaded process has one program counter per thread

Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication

Providing mechanisms for deadlock(A deadlock is a situation in which two computer programs sharing the same resource are effectively preventing each other from accessing the resource, resulting in both programs ceasing to function) handling

Memory Management

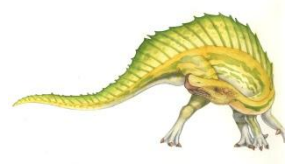
- To execute a program, all (or part) of the instructions must be in _____
- All (or part) of the data that is needed by the program must be in _____
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

Storage Management

- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control
  - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
  - **Privilege escalation** allows user to change to effective ID with more rights



Computing Environments - Traditional

- In 1990's, the “typical office environment consisted of PCs connected to a network, with servers providing file and print services. Remote access was not possible.

Portals provide web access to internal systems.

Network computers (thin clients: A thin client (eg. Client connecting Mail Server) is a terminal that has **no hard drive**. All features typically found on the desktop PC, including applications, sensitive data, memory, etc., are stored back in the data center when using a thin client. Thin clients run an operating system locally and carry flash memory rather than a hard disk.)

Thick client can process data locally – eg. Google Drive

- Mobile computers interconnect via **wireless networks**

- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks

Computing Environments - Mobile

- Handheld smartphones, tablets, etc
- What is the functional difference between them and a "traditional" laptop?
- Extra feature – more OS features (GPS, gyroscope)

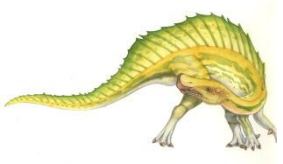
Allows new types of apps like ***augmented reality***

(Augmented reality is the integration of digital information with the user's environment in real time. Unlike virtual reality, which creates a totally artificial environment(it replaces your actual environment with computer generated environment), augmented reality uses the existing environment and overlays new information on top of it)

- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**

Virtual reality has been adopted by the military – this includes all three services (army, navy and air force) – where it is used for training purposes. This is particularly useful for training soldiers for combat situations or other dangerous settings where they have to learn how to react in an appropriate manner.

A virtual reality simulation enables them to do so but without the risk of death or a serious injury. They can re-enact a particular scenario, for example engagement with an enemy in an environment in which they experience this but without the real world risks. This has proven to be safer and less costly than traditional training methods



Computing Environments – Distributed

- Distributed computiing
  - Collection of physically separate, possibly heterogeneous, systems networked together to achieve a common goal.
    - **Network** is a communications path, **TCP/IP** most common

Local Area Network (**LAN**) (connects computers within a room, a building,or a campus)

Wide Area Network (**WAN**) (links buildings, cities, or countries.Eg., A global company may have a WAN to connect its offices worldwide)

Metropolitan Area Network (**MAN**) (link buildings within a city)

- **Personal Area Network** (**PAN**) (Bluetooth)

Network Operating System: An operating system oriented to computer networking, to allow shared file and printer access among multiple computers in a network. This sense is now largely historical, as common operating systems generally now have such features included.

A distributed operating system is an operating system that runs on several machines whose purpose is to provide a useful set of services, generally to make the collection of machines behave more like a single machine. This system looks to its users like an ordinary centralized operating system but runs on multiple, independent central processing units (CPUs).

The distributed operating system plays the same role in making the collective resources of the machines more usable that a typical single-machine operating system plays in making that machine's resources more usable.

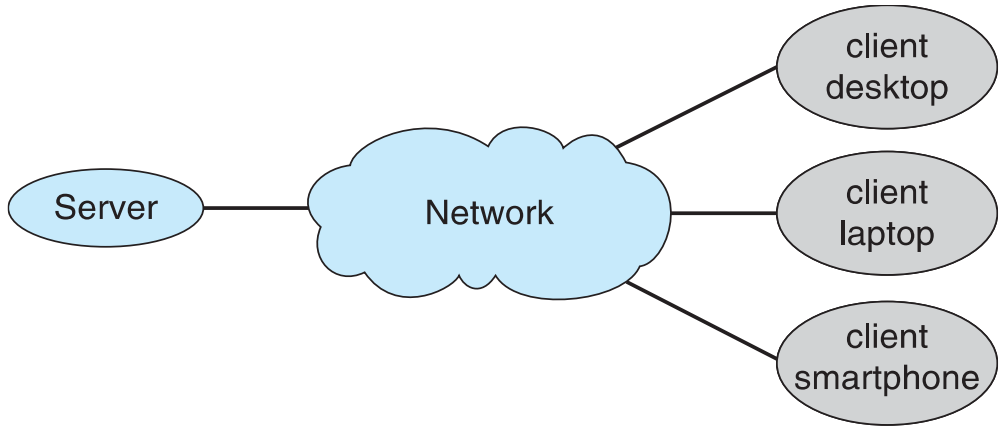
Usually, the machines controlled by a distributed operating system are connected by a relatively high quality network, such as a high speed local area network. Most commonly, the participating nodes of the system are in a relatively small geographical area, something between an office and a campus.

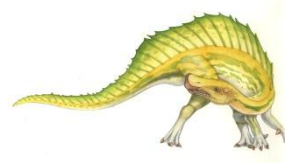
OSes running on the different computers act like a single OS

AEGIS, Spring, SODA

Computing Environments – Client-Server

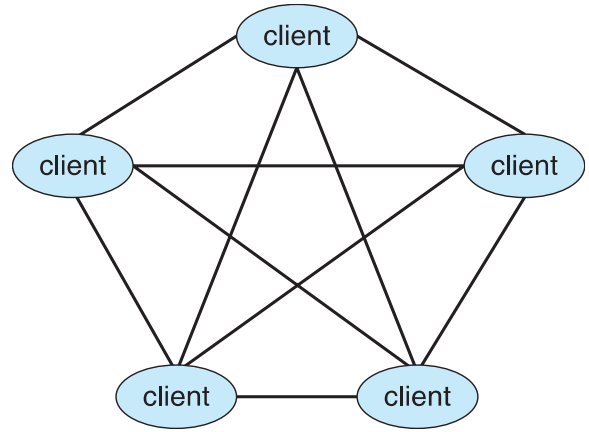
- Client-Server Computing
  - Dumb terminals supplanted by smart PCs
  - Many systems now **servers**, responding to requests generated by **clients**
    - **Compute-server system** provides an interface to client to request services (i.e., database)
    - **File-server system** provides interface for clients to store and retrieve files

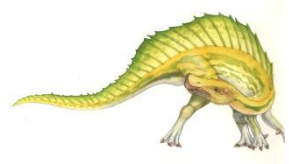




Computing Environments - Peer-to-Peer

- Another model of distributed system
- P2P does not distinguish clients and servers
  - Instead all nodes are considered peers
  - May each act as client, server or both
  - Node must join P2P network
    - Registers its service with central lookup service on network, or
    - Broadcast request for service and respond to requests for service via ***discovery protocol***
  - Examples include Napster and Gnutella, **Voice over IP** (**VoIP**) such as Skype



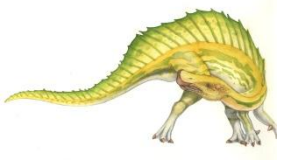


Computing Environments - Virtualization

- Allows operating systems to run applications within other OSes
  - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
  - Generally slowest method
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSes also natively compiled
  - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
  - **VMM** (virtual machine Manager) provides virtualization services

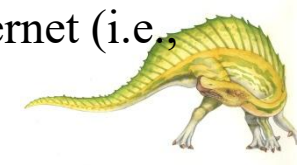
Computing Environments - Virtualization

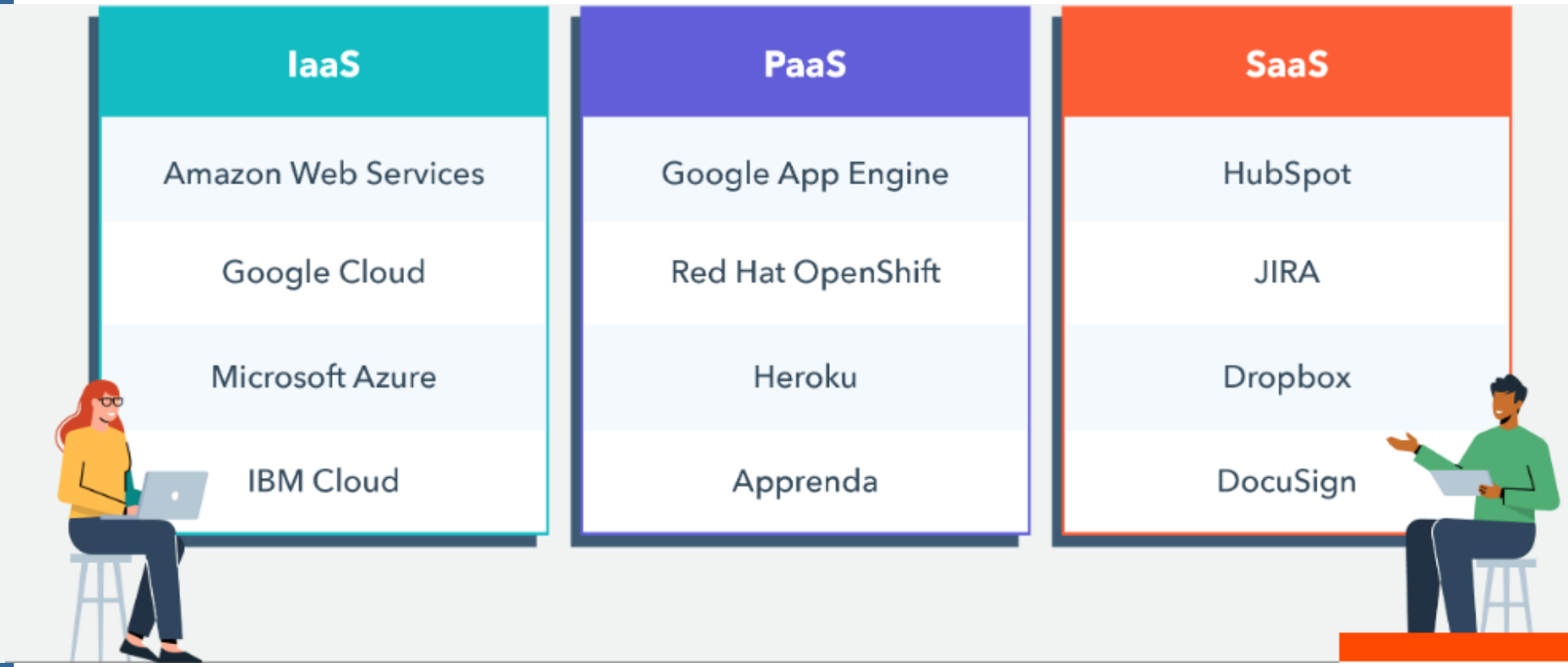
- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
  - Apple laptop running Mac OS X host, Windows as a guest
  - Developing apps for multiple OSes without having multiple systems
  - QA testing applications without having multiple systems
  - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
  - There is no general purpose host then (VMware ESX and Citrix XenServer)

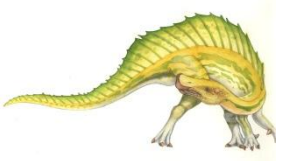


Computing Environments – Cloud Computing

- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for it functionality.
  - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
  - **Public cloud** – available via Internet to anyone willing to pay
  - **Private cloud** – run by a company for the company's own use
  - **Hybrid cloud** – includes both public and private cloud components eg: Azure, Netflix
  - Software as a Service (**SaaS**) – one or more applications available via the Internet (i.e., Microsoft Office 365)
  - Platform as a Service (**PaaS**) – Platform as a service (PaaS) is defined as a cloud computing platform where a third party offers the necessary software and hardware resources (i.e., a database server-Windows Azure)
  - Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (i.e., storage available for backup use, networking, analytics, and compute)

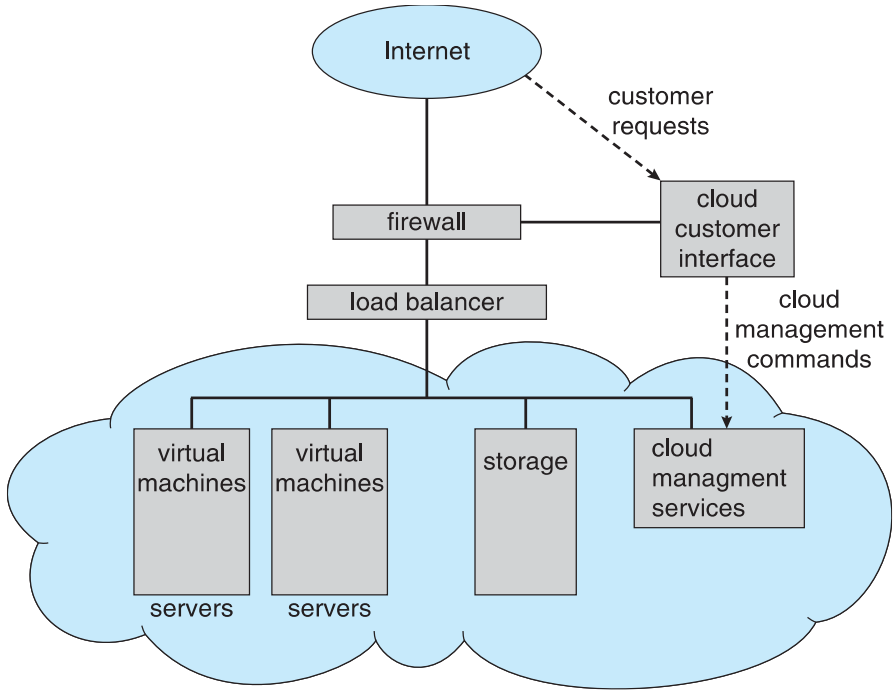


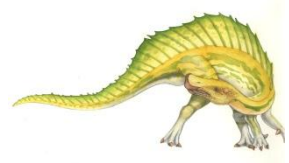

IaaS
Amazon Web Services
Google Cloud
Microsoft Azure
IBM Cloud
PaaS
Google App Engine
Red Hat OpenShift
Heroku
Apprenda
SaaS
HubSpot
JIRA
Dropbox
DocuSign



Computing Environments – Cloud Computing

- Cloud computing environments composed of traditional OSes, plus VMMs, plus cloud management tools
  - Internet connectivity requires security like firewalls
  - Load balancers spread traffic across multiple applications

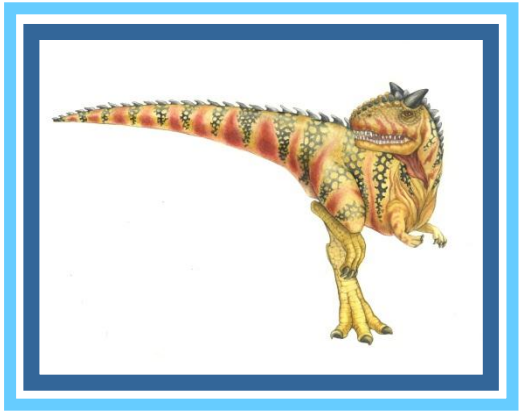




Computing Environments – Real-Time Embedded Systems

- Real-time embedded systems most prevalent form of computers
  - Vary considerable, special purpose, limited purpose OS, **real-time OS**
  - Use expanding
- Many other special computing environments as well
  - Some have OSes, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
  - Processing ***must*** be done within constraint
  - Correct operation only if constraints met

End of Chapter 1



Chapter 2: Operating-System Structures

Chapter 2:

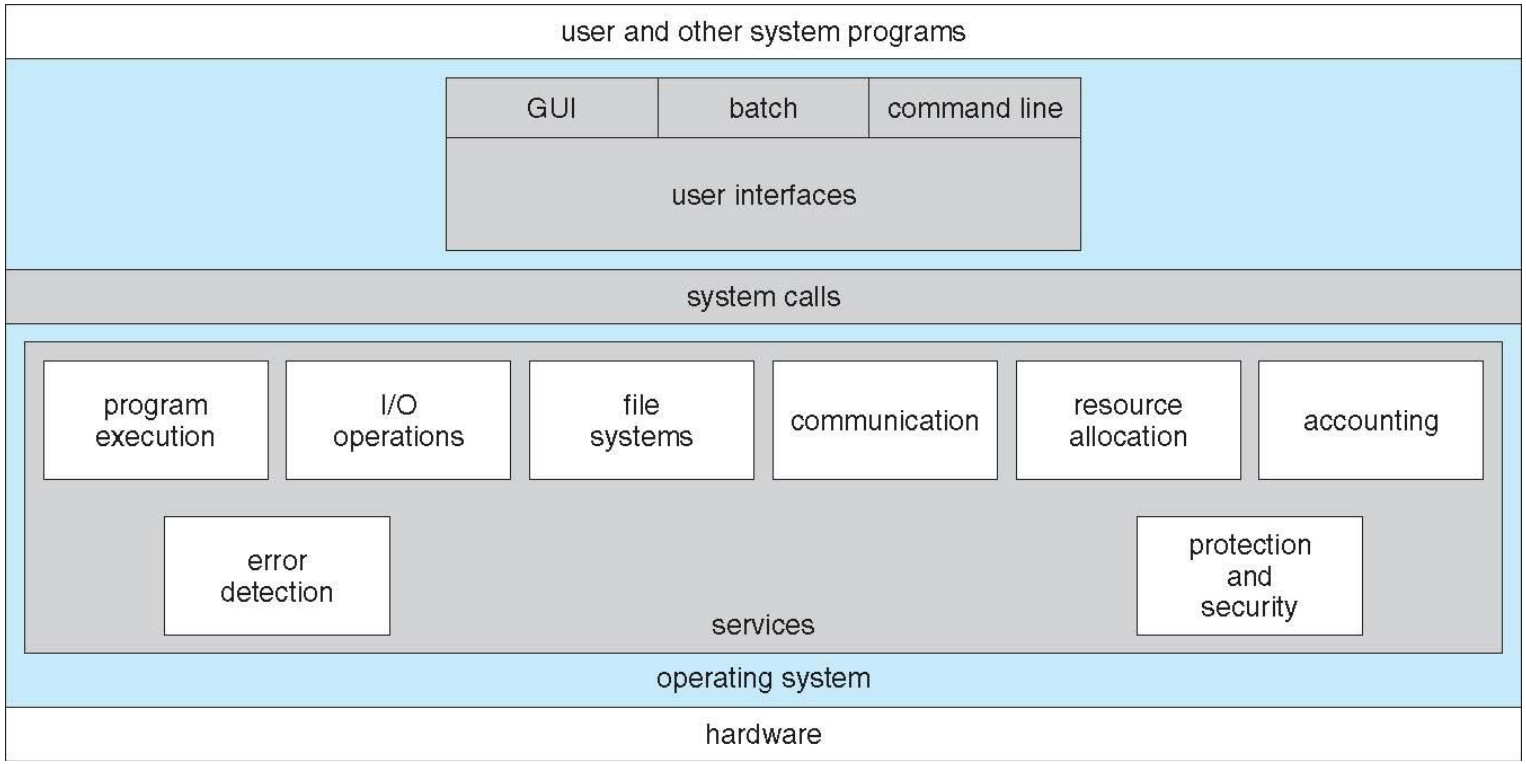
- OS services, system calls,
- system structures,
- virtual machines

(2.1, 2.3, 2.4 – 2.4.1, 2.7-2.7.1, 2.7.2, and 2.7.3)

Objectives

- To describe the services an operating system provides to users, processes, and other systems
- To discuss the various ways of structuring an operating system
- To explain how operating systems are installed and customized and how they boot

A View of Operating System Services



Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device

Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

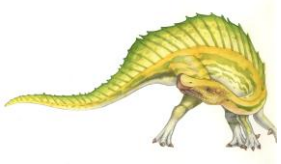
Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Accounting -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

System Calls

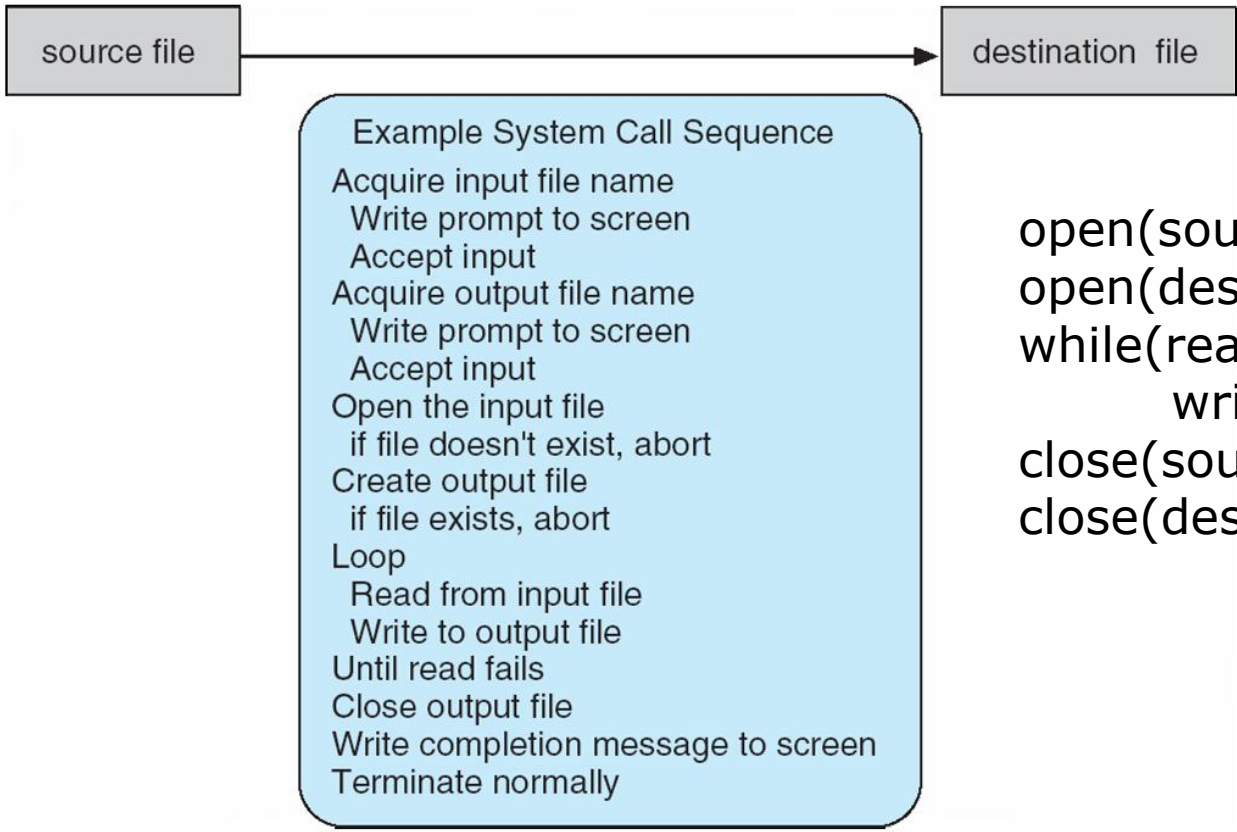
- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic



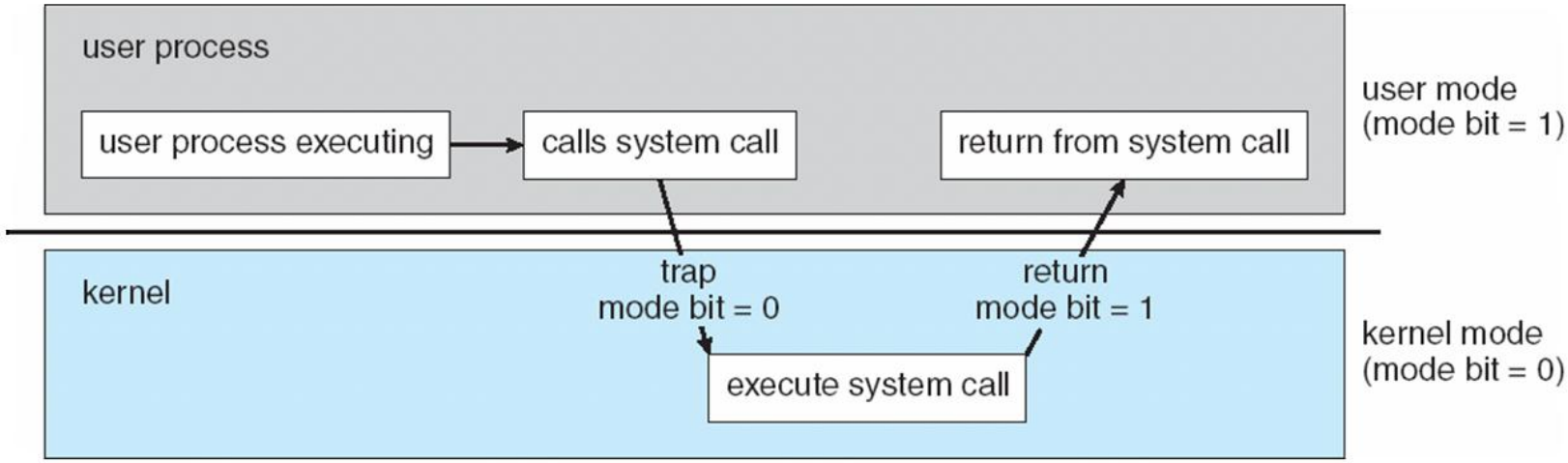
Example of System Calls

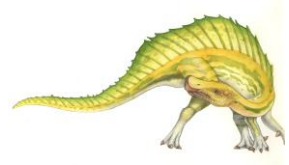
- System call sequence to copy the contents of one file to another file



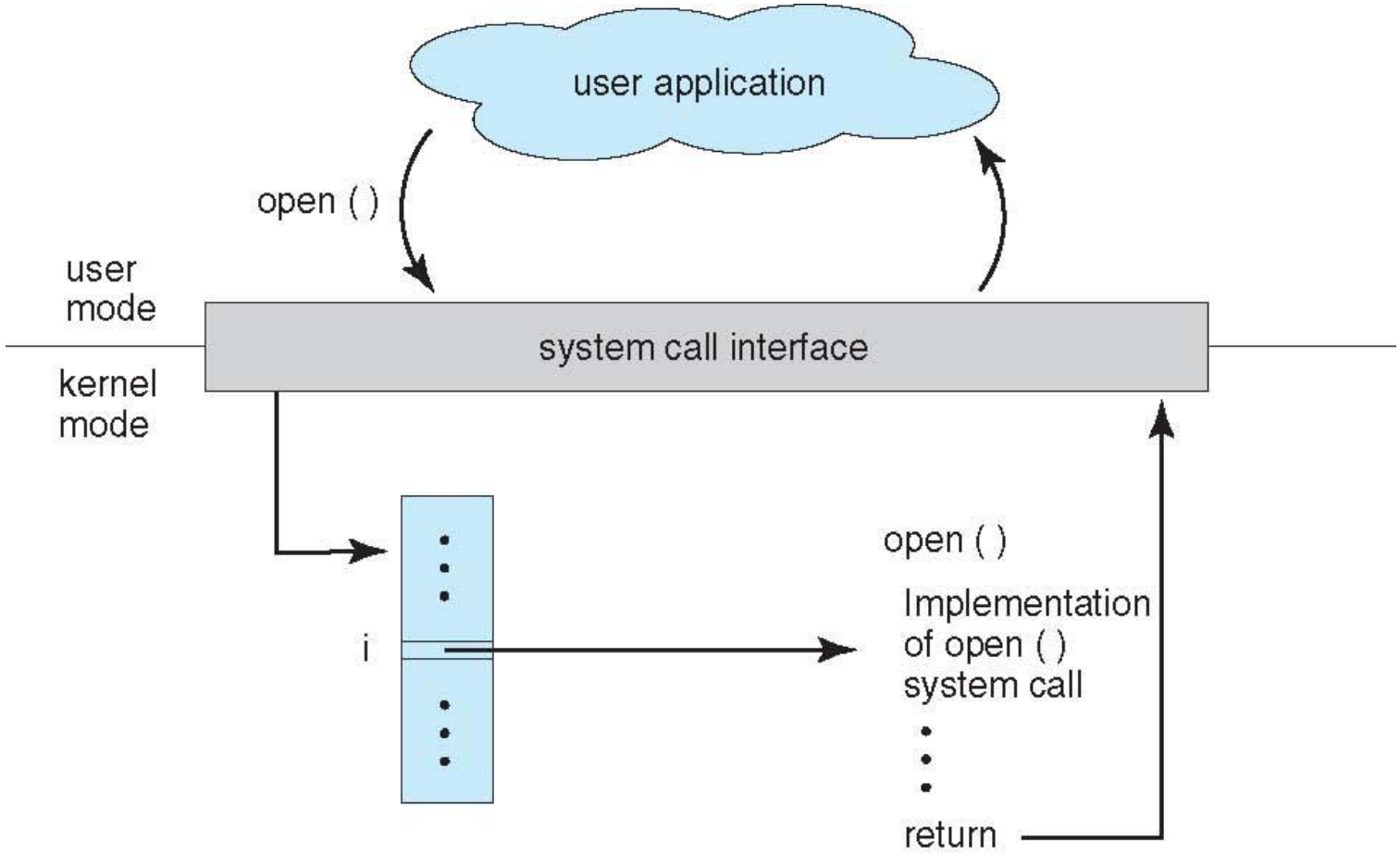

```
open(source)
open(destination)
while(read(source)):
        write(destination)
close(source)
close(destination)
```

Transition from User to Kernel Mode



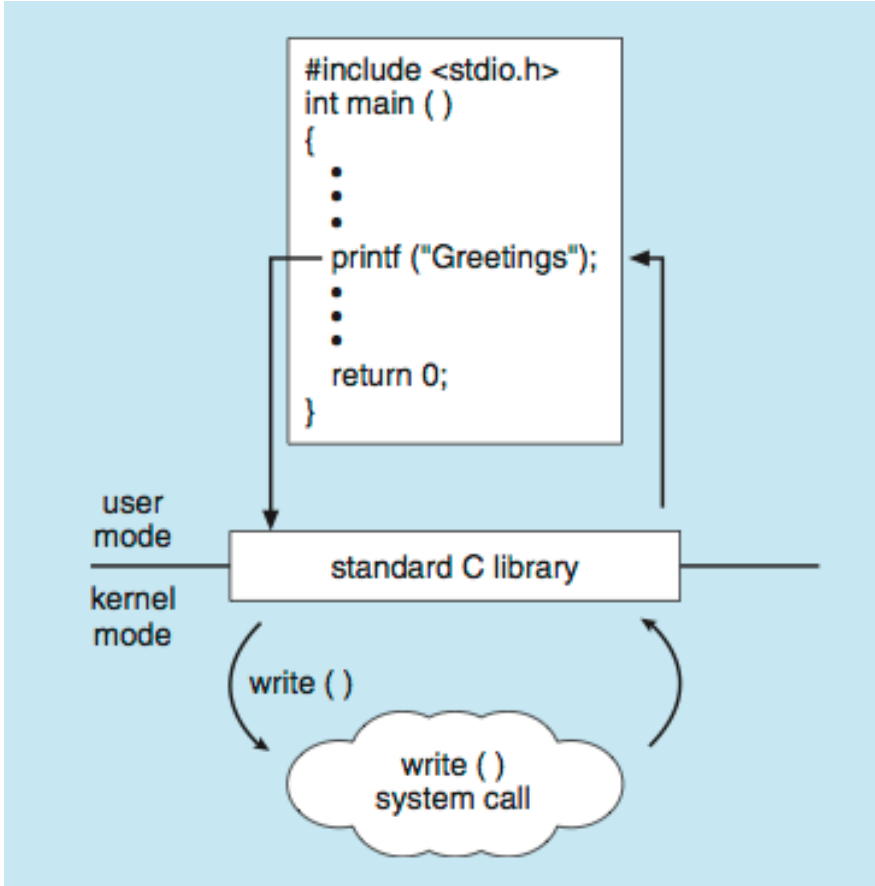


API – System Call – OS Relationship



Standard C Library Example

- C program invoking printf() library call, which calls write() system call



Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t    read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

System Call Implementation

- Typically, a number associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

Advantages of API

- program portability
- actual system calls can often be more detailed and difficult to work
- a strong correlation between a function in the API and its associated system call within the kernel.

System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table

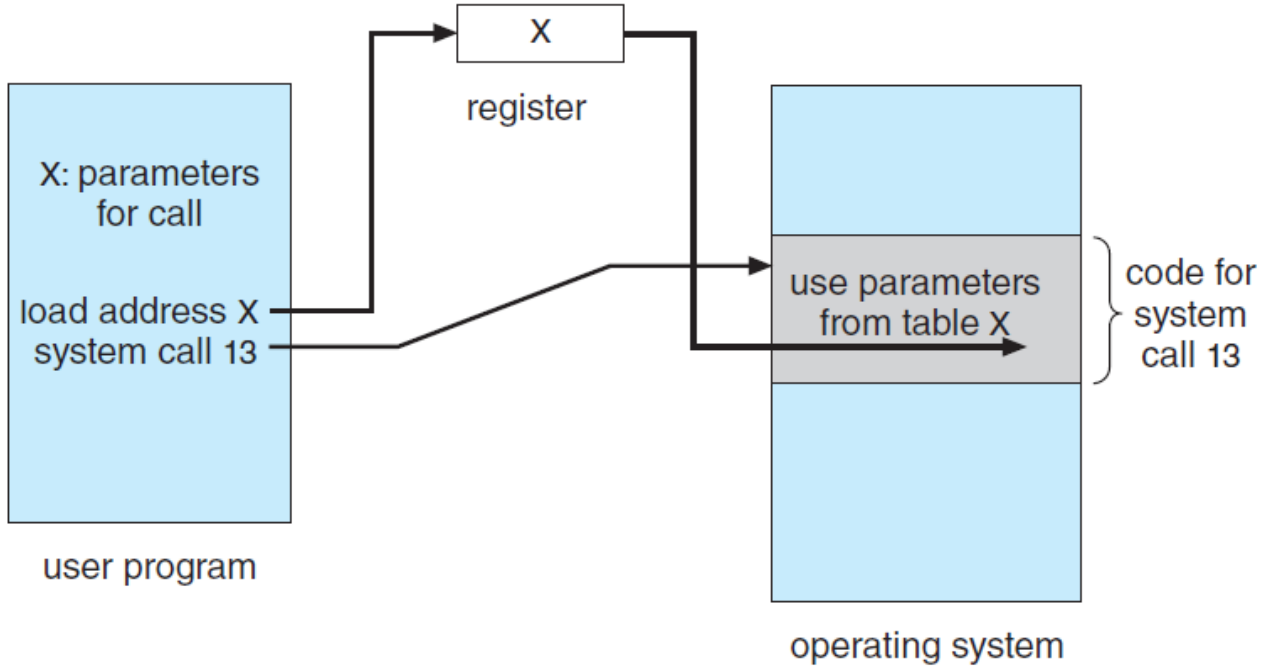
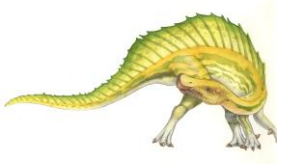


Figure 2.7 Passing of parameters as a table.

Types of System Calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes



Types of System Calls

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

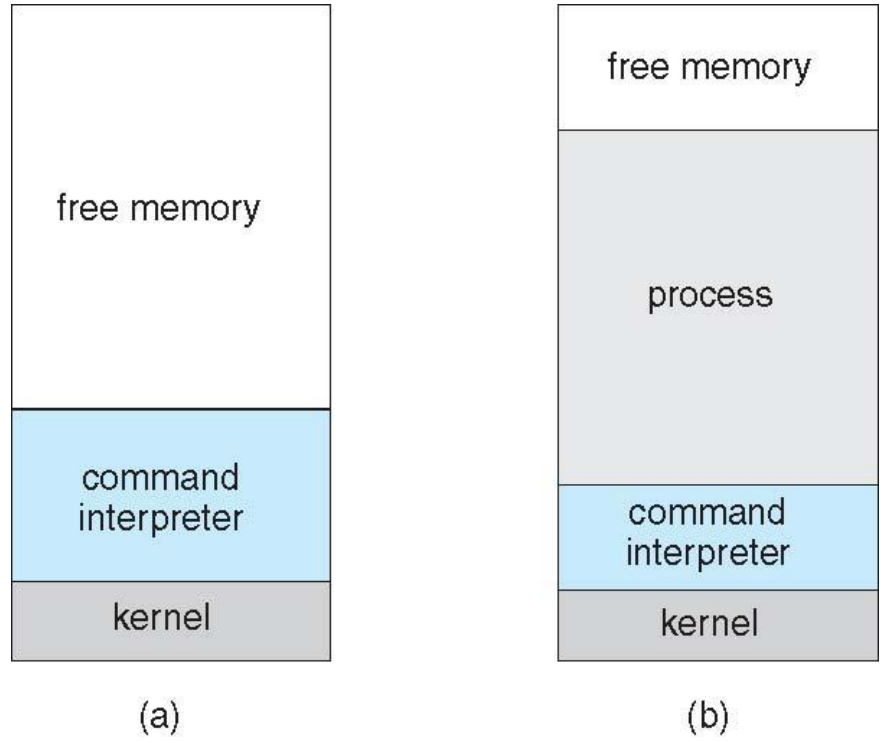
Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Example: MS-DOS

Command interpreter (shell) is handled in memory in an operating system.

- Single-tasking
- Shell invoked when system booted
- Simple method to run program
  - No process created
- Single memory space
- Loads program into memory, overwriting all but the kernel
- Program exit -> shell reloaded



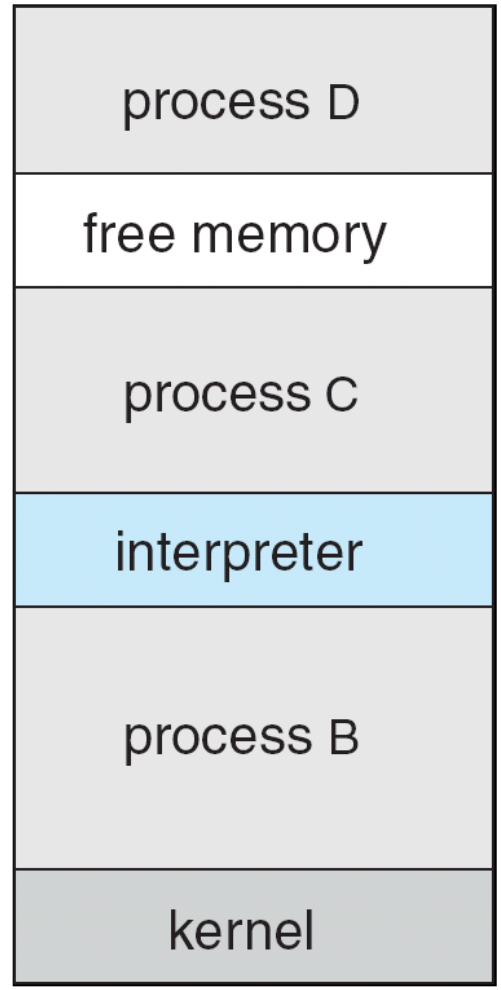
At system startup

running a program

Example: FreeBSD

memory organization in a multiprogramming operating system where multiple processes coexist along with the command interpreter (shell).

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes fork() system call to create process
  - Executes exec() to load program into process
  - Shell waits for process to terminate or continues with user commands
- Process exits with:
  - code = 0 – no error
  - code > 0 – error code



System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information sometimes stored in a File modification
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls

System Programs

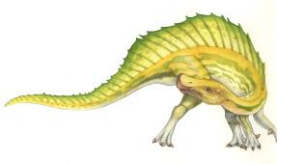
- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information

System Programs (Cont.)

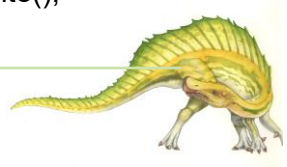
- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

System Programs (Cont.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke



System Program Category	Examples	Purpose	Common System Calls
File Manipulation	ls, cp, mv, rm	Create, delete, copy files	open(), read(), write(), close(), unlink()
Status Information	ps, top, df	Display system status	getpid(), stat(), times()
File Modification	vi, nano, chmod	Modify file content/permissions	read(), write(), chmod()
Programming Language Support	gcc, python, gdb	Compile/interpret programs	fork(), exec(), wait()
Program Loading & Execution	Shell (bash)	Load and execute programs	fork(), exec(), exit()
Communications	pipe, ssh, ftp	Inter-process communication	pipe(), socket(), send(), recv()
Background Services	cron, systemd	Background system services	fork(), sleep(), signal()
Application Programs	Browser, Editor	User applications	open(), read(), write(), mmap()



Operating System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- **Start the design by defining goals and specifications**
- Affected by choice of hardware, type of system
- **User** goals and **System** goals
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

Operating System Design and Implementation (Cont.)

- Important principle to separate

  Policy: ***What*** will be done?
  Mechanism: ***How*** to do it?

- Mechanisms determine how to do something, policies decide what will be done
- Eg:the timer construct is a mechanism for ensuring CPU protection, but deciding how long the timer is to be set for a particular user is a policy decision.
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example – timer)
- Specifying and designing an OS is highly creative task of **software engineering**

Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Modern OS->Actually usually a mix of languages
  - Lowest levels in assembly
  - Kernel->Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware-> Running Linux on Windows using a virtual machine

Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex -- UNIX
  - Layered – an abstraction
  - Microkernel –Mach

Operating System Structure

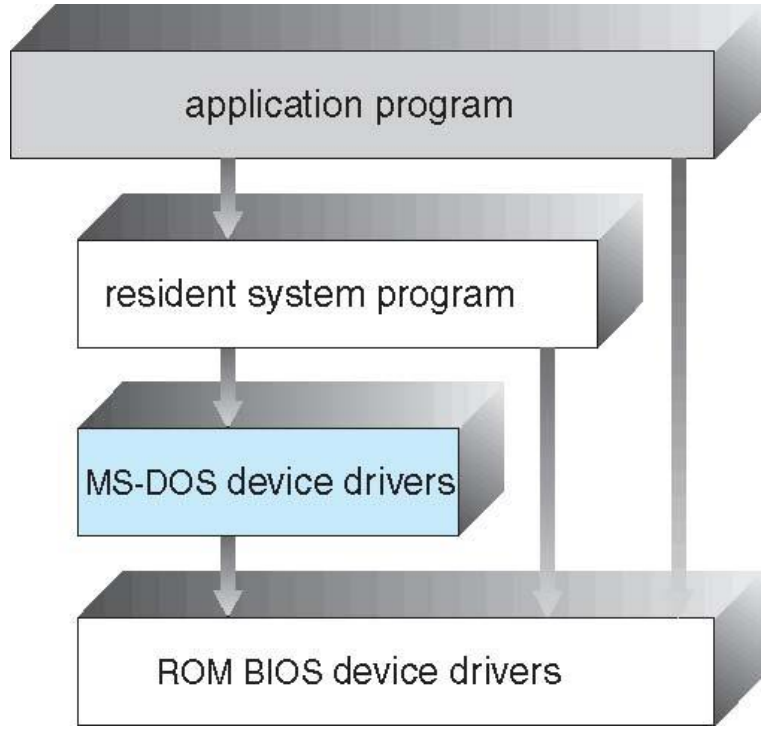
The operating system structure refers to the way in which the various components of an operating system are organized and interconnected.

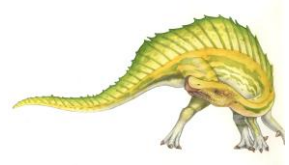
Operating system structure as to how different components of the operating system are interconnected.

- The **internal organization** of an OS differs widely
  - Monolithic kernel
  - Layered architecture
  - Microkernel
  - Modular kernel
  - Hybrid kernel

Simple Structure -- MS-DOS

- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated





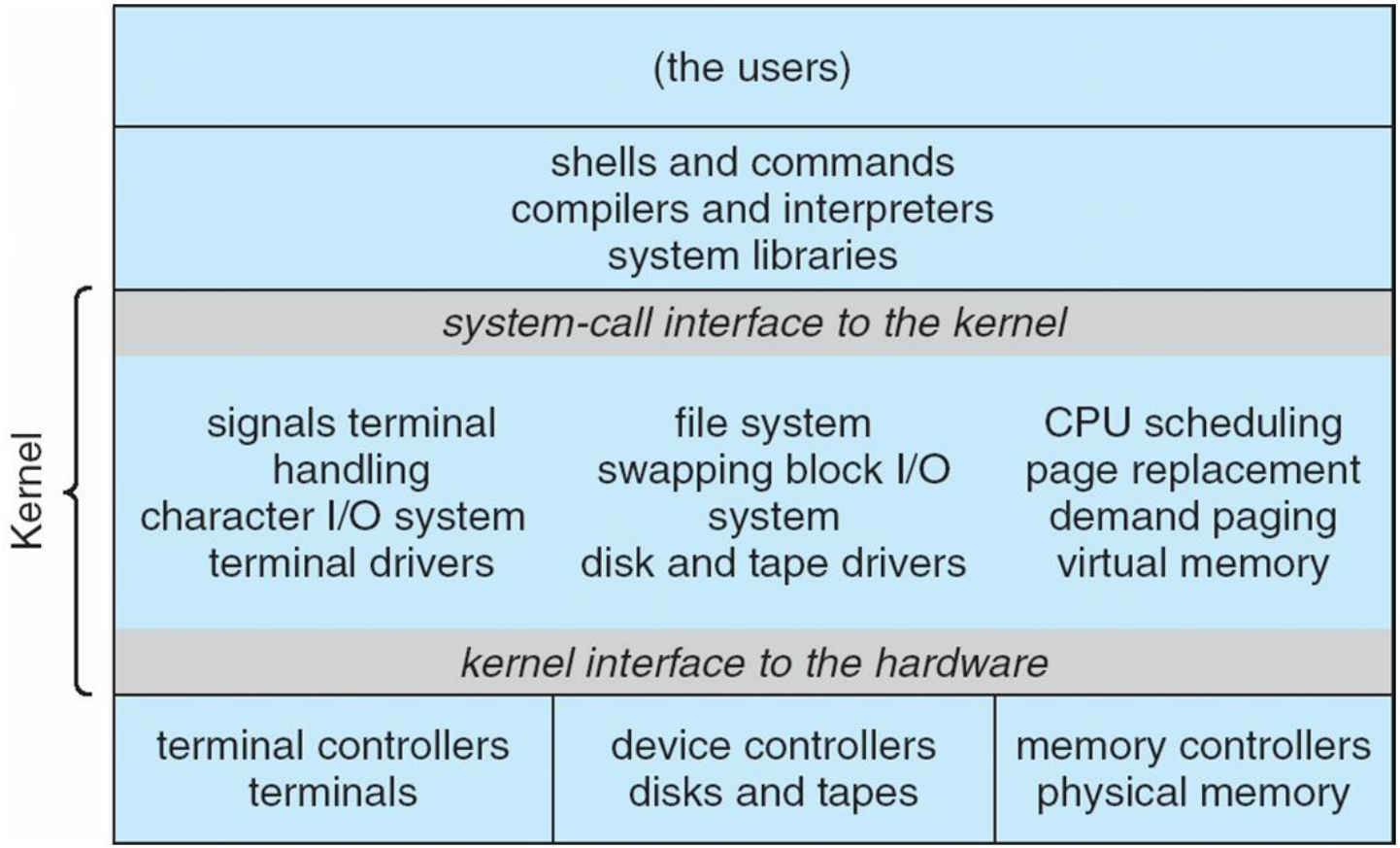
Non Simple Structure -- UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

- Systems programs
- The kernel
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

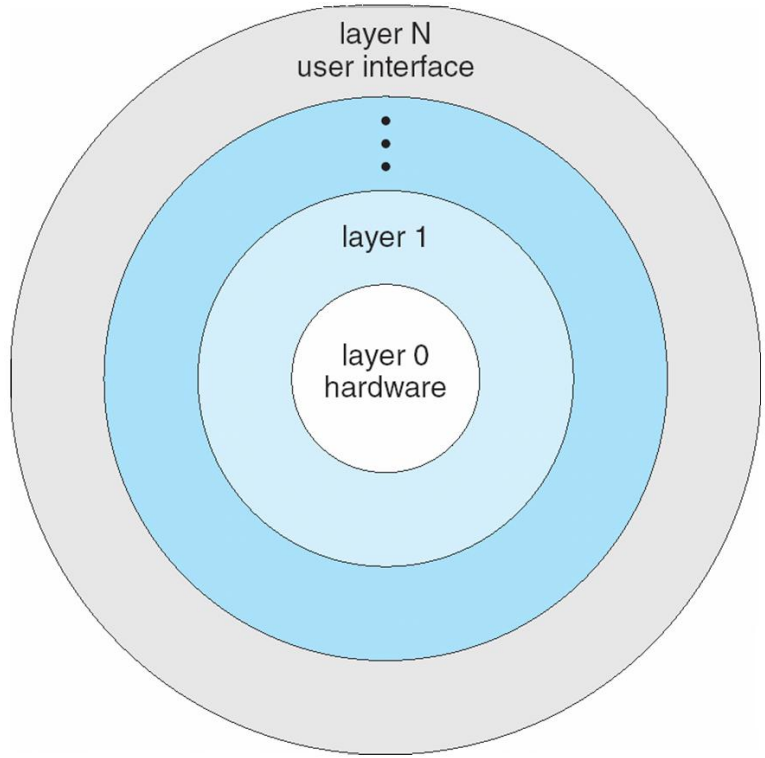
Traditional UNIX System Structure

Beyond simple but not fully layered



Layered Approach

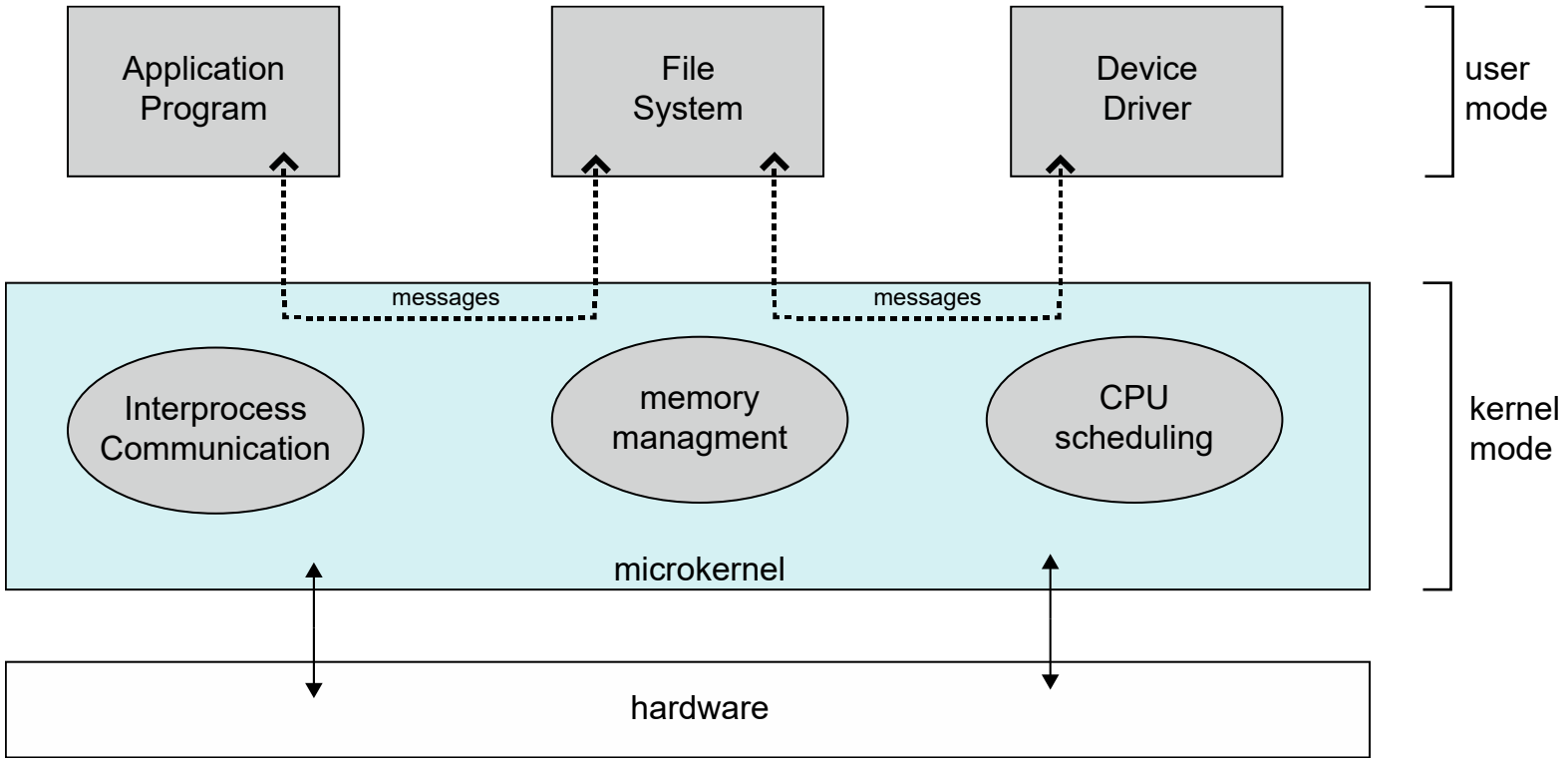
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



Microkernel System Structure

- Moves as much from the kernel into user space
- **Mach** example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

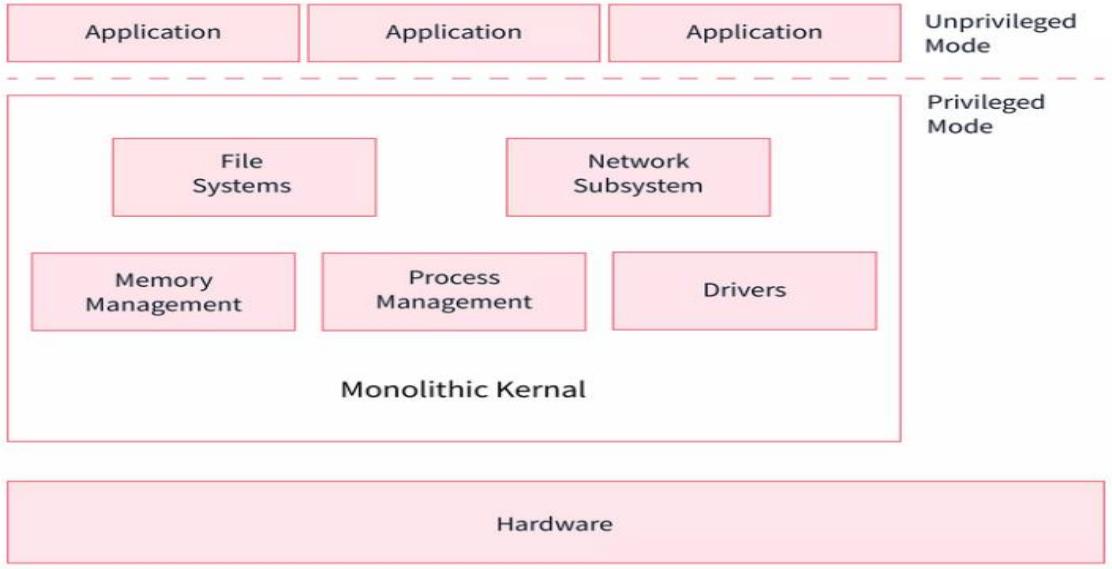
Microkernel System Structure



Monolithic Kernel

The Monolithic operating System in which the kernel acts as a manager by managing all things like file management, memory management, device management, and operational processes of the Operating System.

- In monolithic systems, kernels can directly access all the resources of the operating System like physical hardware, exp Keyboard, Mouse etc.
- **The monolithic kernel is another name for the monolithic operating system. Batch processing and time-sharing maximize the usability of a processor by multiprogramming.**

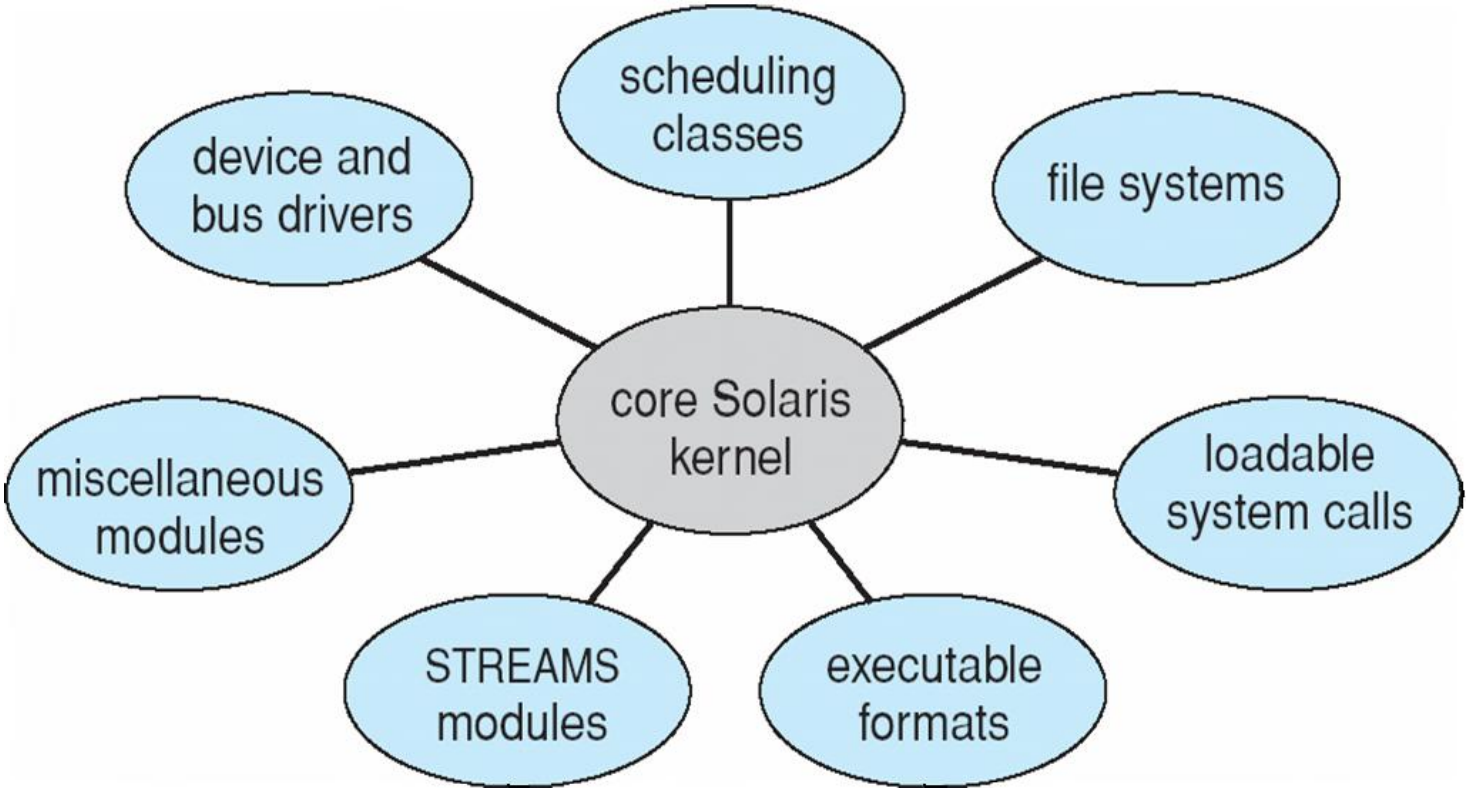


Parameters	Microkernel	Monolithic kernel
Address Space	In microkernel, user services and kernel services are kept in separate address space.	In monolithic kernel, both user services and kernel services are kept in the same address space.
Design and Implementation	OS is complex to design.	OS is easy to design and implement.
Size	Microkernel are smaller in size.	Monolithic kernel is larger than microkernel.
Functionality	Easier to add new functionalities.	Difficult to add new functionalities.
Coding	To design a microkernel, more code is required.	Less code when compared to microkernel
Failure	Failure of one component does not effect the working of micro kernel.	Failure of one component in a monolithic kernel leads to the failure of the entire system.
Processing Speed	Execution speed is low.	Execution speed is high.
Extend	It is easy to extend Microkernel.	It is not easy to extend monolithic kernel.
Communication	To implement IPC messaging queues are used by the communication microkernels.	Signals and Sockets are utilized to implement IPC in monolithic kernels.
Debugging	Debugging is simple.	Debugging is difficult.
Maintain	It is simple to maintain.	Extra time and resources are needed for maintenance.
Message passing and Context switching	Message forwarding and context switching are required by the microkernel.	Message passing and context switching are not required while the kernel is working.
Services	The kernel only offers IPC and low-level device management services.	The Kernel contains all of the operating system's services.
Example	**Example :** Mac OS.	**Example :** Microsoft Windows 95.

Modules

- Many modern operating systems implement **loadable kernel modules**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc

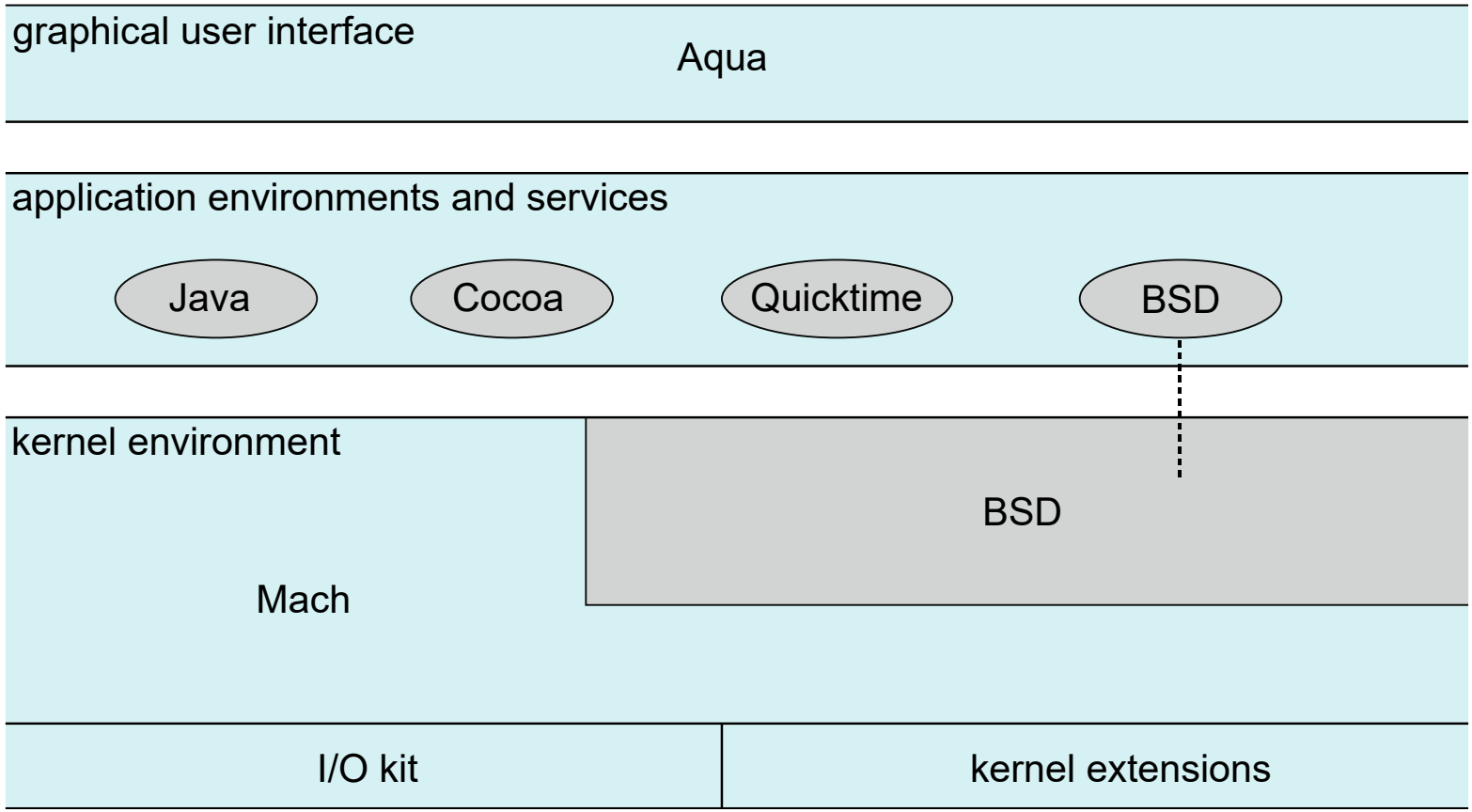
Modular Approach

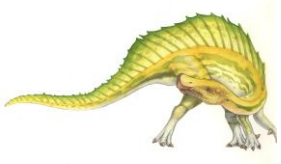


Hybrid Systems

- Most modern operating systems are actually not one pure model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic, plus microkernel for different subsystem ***personalities***
- Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment
  - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)

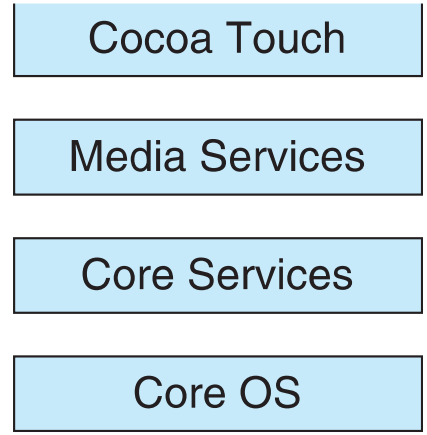
Mac OS X Structure

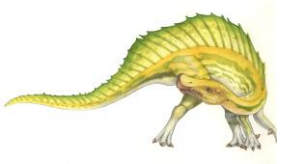




iOS

- Apple mobile OS for ***iPhone***, ***iPad***
  - Structured on Mac OS X, added functionality
  - Does not run OS X applications natively
    - Also runs on different CPU architecture (ARM vs. Intel)
  - **Cocoa Touch** Objective-C API for developing apps
  - **Media services** layer for graphics, audio, video
  - **Core services** provides cloud computing, databases
  - Core operating system, based on Mac OS X kernel





Android

- Developed by Open Handset Alliance (mostly Google)
  - Open Source
- Similar stack to IOS
- Based on Linux kernel but modified
  - Provides process, memory, device-driver management
  - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable than runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc

Android Architecture